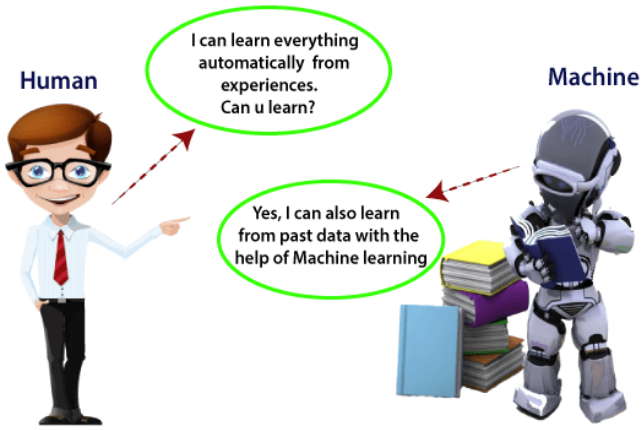
UNIT - I

Introduction: Learning- types of machine learning, Perspectives and issues in machine learning, bias variance trades off, testing machine learning algorithms, confusion matrix, accuracy metrics, turning data into probabilities. Dimensionality reduction- Linear discriminant analysis, principal component analysis, Factor analysis, independent component analysis.

Learning

In the real world, we are surrounded by humans who can learn everything from their experiences with their learning capability, and we have computers or machines which work on our instructions. But can a machine also learn from experiences or past data like a human does? So here comes the role of **Machine Learning**.



Machine Learning

Arthur Samuel first used the term "machine learning" in 1959.

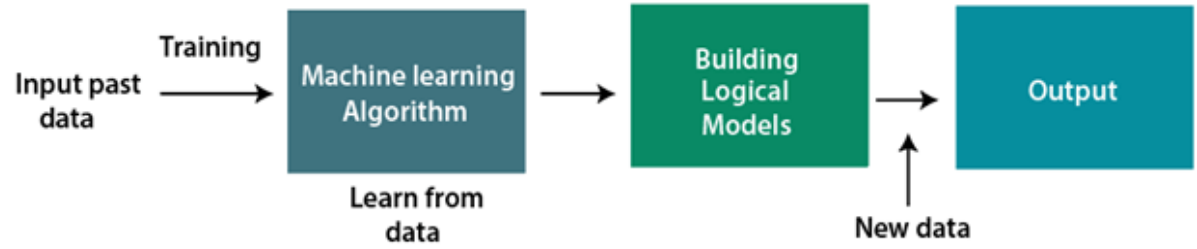
Machine learning is a subfield of artificial intelligence (AI) that uses algorithms trained on data sets to create self-learning models that are capable of predicting outcomes and classifying information without human intervention.

Machine learning is used today for a wide range of commercial purposes, including suggesting products to consumers based on their past purchases, predicting stock market fluctuations, and translating text from one language to another.

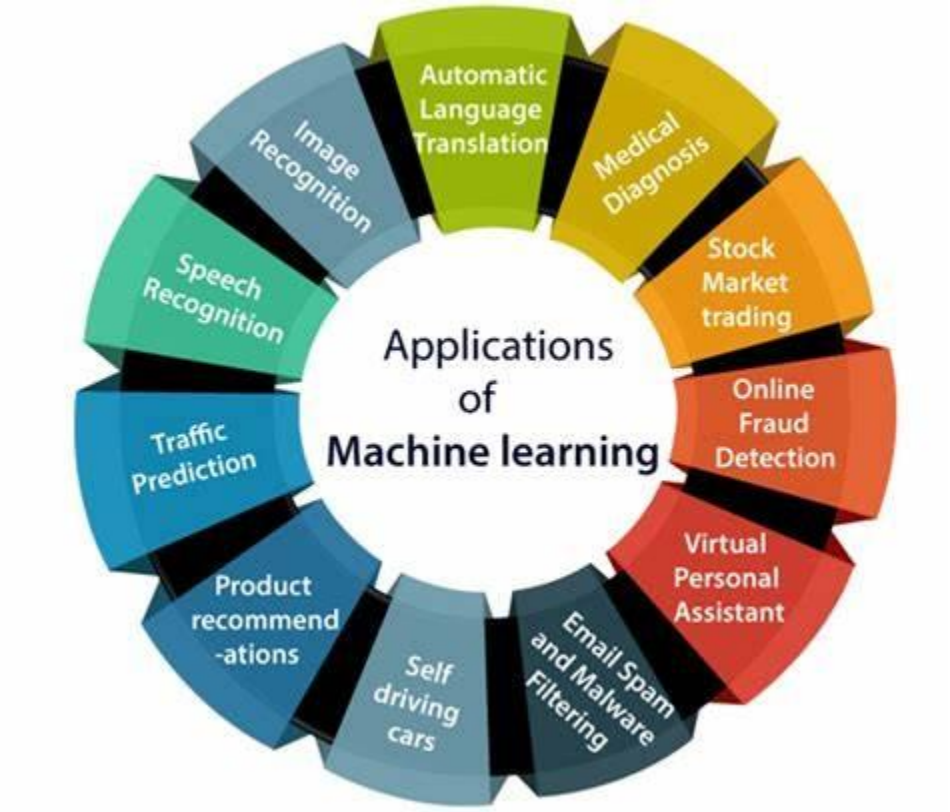
Some of the most common examples of machine learning that you may have interacted with in your day-to-day life include:

- Recommendation engines that suggest products, songs, or television shows to you, such as those found on Amazon, Spotify, or Netflix.
- Speech recognition software that allows you to convert voice memos into text.
- A bank’s fraud detection services automatically flag suspicious transactions.
- Self-driving cars and driver assistance features, such as blind-spot detection and automatic stopping, improve overall vehicle safety.

How machine learning work?



Machine learning contains a set of algorithms that work on a huge amount of data. Data is fed to these algorithms to train them, and on the basis of training, they build the model & perform a specific task.



TYPES OF MACHINE LEARNING

Based on the methods and way of learning, machine learning is divided into mainly four types, which are:

1. Supervised Machine Learning
2. Unsupervised Machine Learning
3. Reinforcement Learning

1. Supervised Machine Learning

Imagine a teacher supervising a class. The teacher already knows the correct answers but the learning process doesn't stop until the students learn the answers as well. This is the essence of Supervised Machine Learning Algorithms.

Supervised learning is where the model is trained on a labelled dataset. A labelled dataset is one that has both input and output parameters. In this type of learning both training and validation, datasets are labelled as shown in the figures below.

The labelled dataset used in supervised learning consists of input features and corresponding output labels. The input features are the attributes or characteristics of the data that are used to make predictions, while the output labels are the desired outcomes or targets that the algorithm tries to predict.

User ID	Gender	Age	Salary	Purchased
15624510	Male	19	19000	0
15810944	Male	35	20000	1
15668575	Female	26	43000	0
15603246	Female	27	57000	0
15804002	Male	19	76000	1
15728773	Male	27	58000	1
15598044	Female	27	84000	0
15694829	Female	32	150000	1
15600575	Male	25	33000	1
15727311	Female	35	65000	0
15570769	Female	26	80000	1
15606274	Female	26	52000	0
15746139	Male	20	86000	1
15704987	Male	32	18000	0
15628972	Male	18	82000	0
15697686	Male	29	80000	0
15733883	Male	47	25000	1

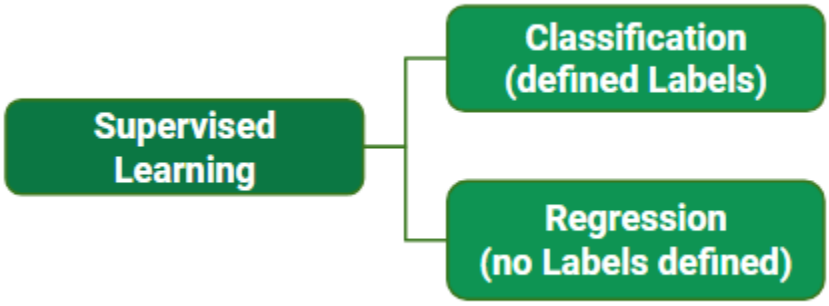
Figure A: CLASSIFICATION

Figure A: It is a dataset of a shopping store that is useful in predicting whether a customer will purchase a particular product under consideration or not based on his/ her gender, age, and salary.

Input: Gender, Age, Salary

Output: Purchased i.e. 0 or 1; 1 means yes the customer will purchase and 0 means that the customer won't purchase it.

Types of Supervised Learning Algorithm



Regression

Regression is a type of supervised machine learning where algorithms learn from the data to predict continuous values such as sales, salary, weight, or temperature. For example:

A dataset containing features of the house such as lot size, number of bedrooms, number of baths, neighborhood, etc. and the price of the house, a Regression algorithm can be trained to learn the relationship between the features and the price of the house.

There are many machine learning algorithms that can be used for regression tasks. Some of them are:

- Linear Regression
- Decision Tree
- K Nearest Neighbor
- Random Forest
- Neural Networks



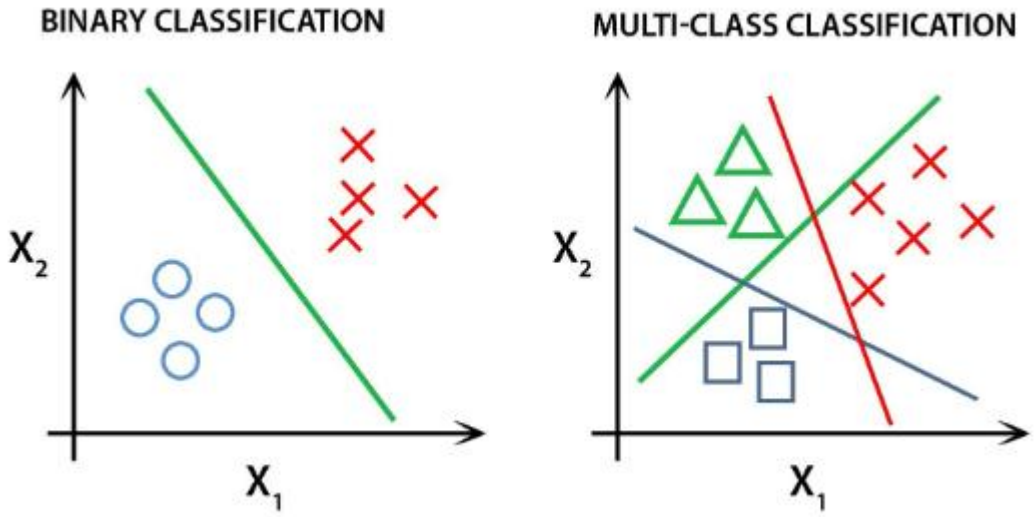
Classification

Classification is a type of supervised machine learning where algorithms learn from the data to predict an outcome or event in the future. For example:

A bank may have a customer dataset containing credit history, loans, investment details, etc. and they may want to know if any customer has paid loan in time or not etc . In the historical data, we will have Features and Target.

- Features will be attributes of a customer such as credit history, loans, investments, etc.

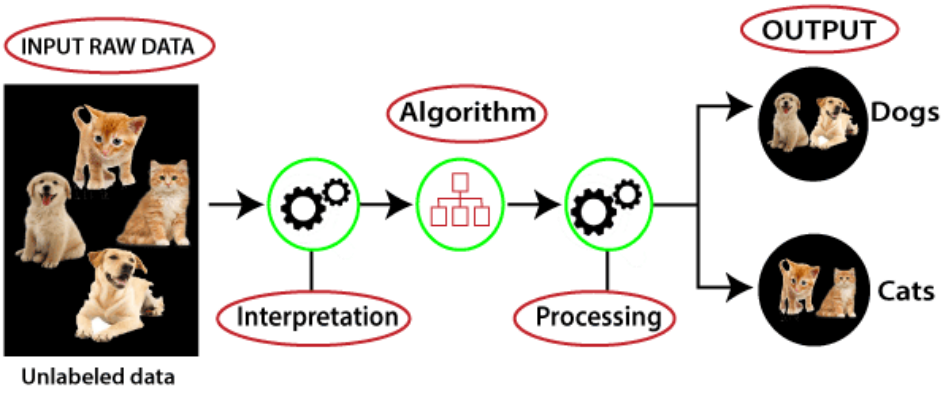
- Target will represent whether a particular customer has paid in the past or not. (normally represented by 1 or 0 / True or False / Yes or No.
  Classification algorithms are used for predicting discrete outcomes, if the outcome can take two possible values such as True or False, Default or No Default, Yes or No, it is known as Binary Classification. When the outcome contains more than two possible values, it is known as Multiclass Classification. There are many machine learning algorithms that can be used for classification tasks. Some of them are:
- Logistic Regression
- Decision Tree Classifier
- K Nearest Neighbor Classifier
- Random Forest Classifier
- Neural Networks



2. Unsupervised Machine Learning

Unsupervised Learning is a machine learning technique in which the users do not need to supervise the model. Instead, it allows the model to work on its own to discover patterns and information that was previously undetected. It mainly deals with the unlabelled data. Unsupervised learning algorithms include clustering, anomaly detection, neural networks, etc.

- Unsupervised learning is much similar as a human learns to think by their own experiences, which makes it closer to the real AI.
- Unsupervised learning works on unlabeled and uncategorized data which make unsupervised learning more important.
- In real-world, we do not always have input data with the corresponding output so to solve such cases, we need unsupervised learning.
- Working of unsupervised learning can be understood by the below diagram:



- Here, we have taken an unlabeled input data, which means it is not categorized and corresponding outputs are also not given. Now, this unlabeled input data is fed to the machine learning model in order to train it. Firstly, it will interpret the raw data to find the hidden patterns from the data and then will apply suitable algorithms such as k-means clustering, Decision tree, etc.
- Once it applies the suitable algorithm, the algorithm divides the data objects into groups according to the similarities and difference between the objects.

Unsupervised learning algorithms:

- **K-means clustering**
- **KNN (k-nearest neighbors)**
- **Hierarchal clustering**
- **Neural Networks**
- **Apriori algorithm**

Advantages of Unsupervised Learning

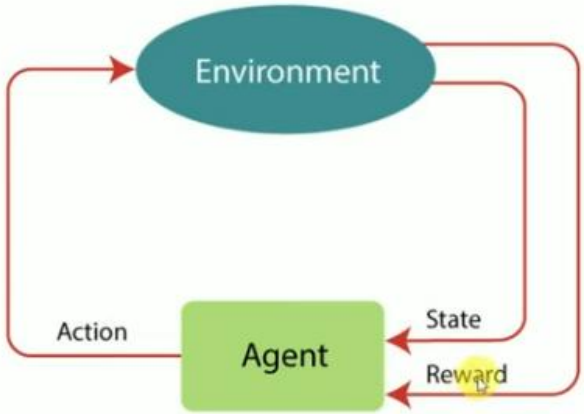
- Unsupervised learning is used for more complex tasks as compared to supervised learning because, in unsupervised learning, we don't have labeled input data.
- Unsupervised learning is preferable as it is easy to get unlabeled data in comparison to labeled data.

Disadvantages of Unsupervised Learning

- Unsupervised learning is intrinsically more difficult than supervised learning as it does not have corresponding output.
- The result of the unsupervised learning algorithm might be less accurate as input data is not labelled, and algorithms do not know the exact output in advance.

3. Reinforcement Learning

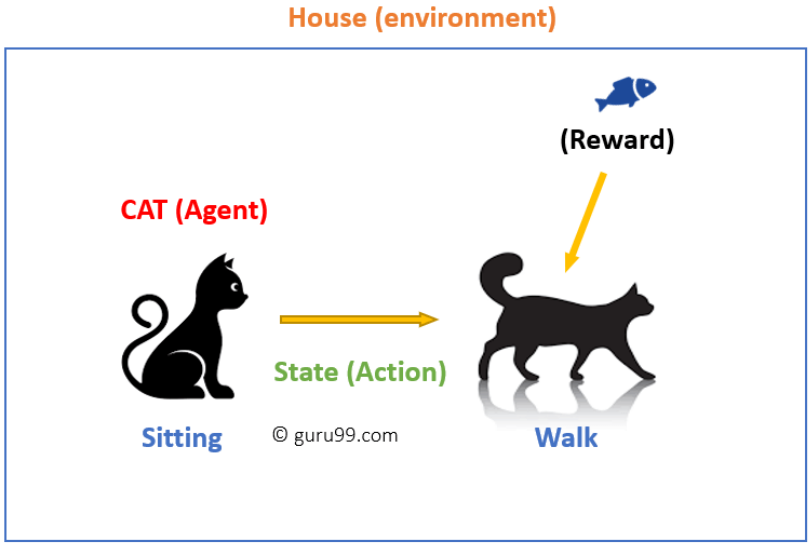
- Reinforcement Learning is a feedback-based Machine learning Approach here an agent learns to which actions to perform by looking at the environment and the results of actions.
- For each correct action, the agent gets positive feedback, and for each incorrect action, the agent gets negative feedback or penalty.



Example:

Consider the scenario of teaching new tricks to your cat

- As cat doesn't understand English or any other human language, we can't tell her directly what to do. Instead, we follow a different strategy.
- We emulate a situation, and the cat tries to respond in many different ways. If the cat's response is the desired way, we will give her fish.
- Now whenever the cat is exposed to the same situation, the cat executes a similar action with even more enthusiastically in expectation of getting more reward(food).
- That's like learning that cat gets from "what to do" from positive experiences.
- At the same time, the cat also learns what not do when faced with negative experiences.



How Reinforcement Learning works

In this case,

- Your cat is an agent that is exposed to the environment. In this case, it is your house. An example of a state could be your cat sitting, and you use a specific word in for cat to walk.

- Our agent reacts by performing an action transition from one “state” to another “state.”
- For example, your cat goes from sitting to walking.
- The reaction of an agent is an action, and the policy is a method of selecting an action given a state in expectation of better outcomes.
- After the transition, they may get a reward or penalty in return.

Example: Consider that you are training an AI agent to play a game like chess. The agent explores different moves and receives positive or negative feedback based on the outcome. Reinforcement Learning also finds applications in which they learn to perform tasks by interacting with their surroundings.

Issues in machine learning:

There are a lot of challenges that machine learning professionals face to inculcate ML skills and create an application from scratch. What are these challenges?

1**. Poor Quality of Data**

Data plays a significant role in the machine learning process. One of the significant issues that machine learning professionals face is the absence of good quality data. Unclean and noisy data can make the whole process extremely exhausting. We don’t want our algorithm to make inaccurate or faulty predictions. Hence the quality of data is essential to enhance the output. Therefore, we need to ensure that the process of data pre-processing which includes removing outliers, filtering missing values, and removing unwanted features, is done with the utmost level of perfection.

2. **Under fitting of Training Data**

Underfitting is the opposite of overfitting. It happens when a model is too simple to capture what’s going on in the data.

- For example, imagine drawing a straight line to fit points that actually follow a curve. The line misses most of the pattern.
- In this case, the model doesn’t work well on either the training or testing data.

Underfitting models are like students who don’t study enough. They don’t do well in practice tests or real exams.

Note: The underfitting model has High bias and low variance.

To overcome this issue:

- Enhance the complexity of the model
- Add more features to the data
- Increasing the training time of model

3. **Overfitting of Training Data**

Overfitting happens when a model learns too much from the training data, including details that don’t matter (like noise or outliers).

For example, imagine fitting a very complicated curve to a set of points. The curve will go through every point, but it won’t represent the actual pattern.

As a result, the model works great on training data but fails when tested on new data.

Overfitting models are like students who memorize answers instead of understanding the topic. They do well in practice tests (training) but struggle in real exams (testing).

Example to Understand Overfitting

We can understand overfitting with a general example. Suppose there are three students, X, Y, and Z, and all three are preparing for an exam. X has studied only three sections of the book and left all other sections. Y has a good memory, hence memorized the whole book. And the third student, Z, has studied and practiced all the questions. So, in the exam, X will only be able to solve the questions if the exam has questions related to section 3. Student Y will only be able to solve questions if they appear exactly the same as given in the book. Student Z will be able to solve all the exam questions in a proper way.

The same happens with machine learning; if the algorithm learns from a small part of the data, it is unable to capture the required data points and hence under fitted.

Suppose the model learns the training dataset, like the Y student. They perform very well on the seen dataset but perform badly on unseen data or unknown instances. In such cases, the model is said to be Overfitting.

And if the model performs well with the training dataset and also with the test/unseen dataset, similar to student Z, it is said to be a good fit.

We can tackle this issue by:

- Analysing the data with the utmost level of perfection
- Use data augmentation technique
- Remove outliers in the training set
- Select a model with lesser features

4. Machine Learning is a Complex Process

The machine learning industry is young and is continuously changing. Rapid hit and trial experiments are being carried on. The process is transforming, and hence there are high chances of error which makes the learning complex**. It includes analyzing the data, removing data bias, training data, applying complex mathematical calculations, and a lot more.** Hence it is a really complicated process which is another big challenge for Machine learning professionals.

5. Lack of Training Data

The most important task you need to do in the machine learning process is to train the data to achieve an accurate output. Less amount training data will produce inaccurate or too biased predictions. Let us understand this with the help of an example. Consider a machine learning algorithm similar to training a child.

One day you decided to explain to a child how to distinguish between an apple and a watermelon. You will take an apple and a watermelon and show him the difference between both based on their color, shape, and taste. In this way, soon, he will attain perfection in differentiating between the two.

But on the other hand, a machine-learning algorithm needs a lot of data to distinguish. For complex problems, it may even require millions of data to be trained. Therefore we need to ensure that Machine learning algorithms are trained with sufficient amounts of data.

6. Slow Implementation

This is one of the common issues faced by machine learning professionals. The machine learning models are highly efficient in providing accurate results, but it takes a **tremendous amount of time**. Slow programs, data overload, and excessive requirements usually take a lot of time to provide accurate results. Further, it requires constant monitoring and maintenance to deliver the best output.

7. Imperfections in the Algorithm When Data Grows

We have found quality data, trained it amazingly, and the predictions are really concise and accurate. But wait, there is a twist; the model may become useless in the future as data grows. The best model of the present may become inaccurate in the coming Future and require further rearrangement. **So you need regular monitoring and maintenance** to keep the algorithm working. This is one of the most exhausting issues faced by machine learning professionals.

Bias-Variance Trade off

While discussing model accuracy, we need to keep in mind the **prediction errors, ie: Bias and Variance,** that will always be associated with any machine learning model. There will always be a slight difference in what our model predicts and the actual predictions. These differences are called errors. The goal of an analyst is not to eliminate errors but to reduce them. There is always a tradeoff between how low you can get errors to be.

Errors in Machine Learning

We can describe an error as an action which is inaccurate or wrong. In Machine Learning, error is used to see how accurately our model can predict on data it uses to learn; as well as new, unseen data. Based on our error, we choose the machine learning model which performs best for a particular dataset.

There are two main types of errors present in any machine learning model. They are Reducible Errors and Irreducible Errors.

- **Irreducible errors** are errors which will always be present in a machine learning model, because of unknown variables, and whose values cannot be reduced.
- **Reducible errors** are those errors whose values can be further reduced to improve a model. They are caused because our model's output function does not match the desired output function and can be optimized.

We can further divide reducible errors into two: Bias and Variance.

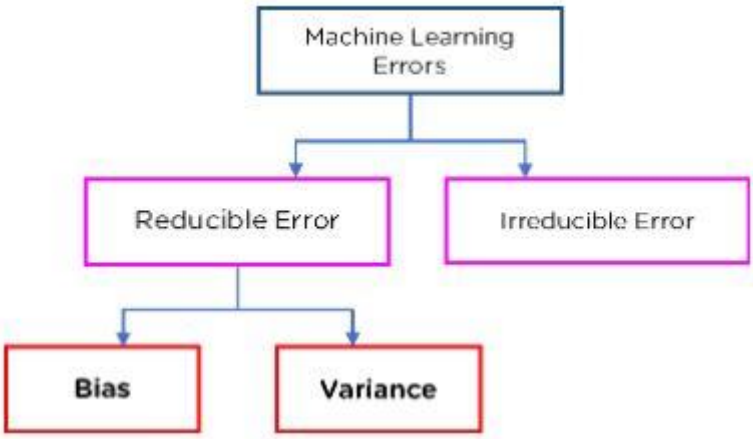


Figure 1: Errors in Machine Learning

What is Bias?

- To make predictions, our model will analyze our data and find patterns in it. Using these patterns, we can make generalizations about certain instances in our data. Our model after training learns these patterns and applies them to the test set to predict them.
- Bias is the difference between our actual and predicted values. Bias is the simple assumptions that our model makes about our data to be able to predict new data.

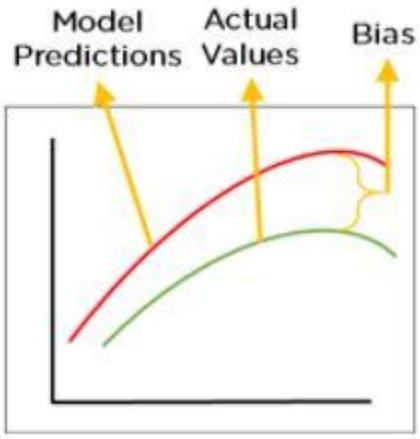


Figure 2: Bias

When the Bias is high, assumptions made by our model are too basic, the model can't capture the important features of our data. **This means that our model hasn't captured patterns in the training data and hence cannot perform well on the testing data too.** If this is the case, our model cannot perform on new data and cannot be sent into production.

This instance, where the model cannot find patterns in our training set and hence fails for both seen and unseen data, is called Underfitting.

The below figure shows an example of Underfitting. As we can see, the model has found no patterns in our data and the line of best fit is a straight line that does not pass through any of the data points. The model has failed to train properly on the data given and cannot predict new data either.

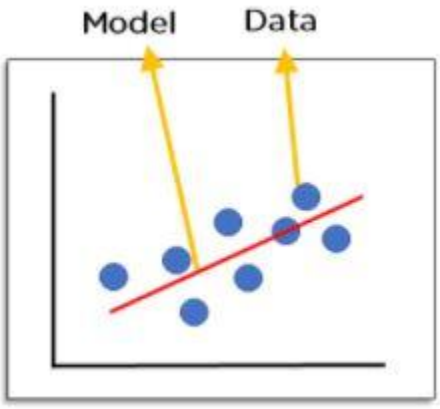


Figure 3: Underfitting

What is Variance?

Variance is the very opposite of Bias. During training, it allows our model to 'see' the data a certain number of times to find patterns in it. If it does not work on the data for long enough, it will not find patterns and bias occurs. On the other hand, if our model is allowed to view the data too many times, it will learn very well for only that data. It will capture most patterns in the data, but it will also learn from the unnecessary data present, or from the noise.

We can define variance as the model's sensitivity to fluctuations in the data. Our model may learn from noise. This will cause our model to consider trivial features as important.

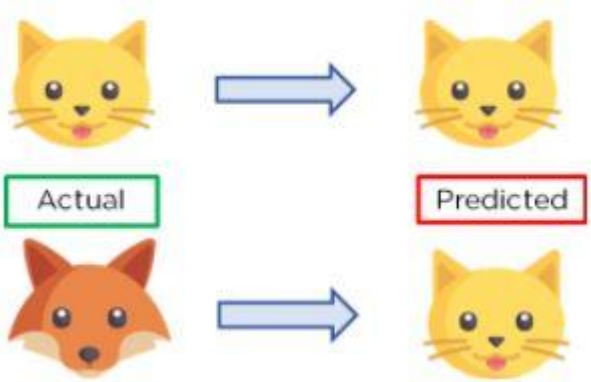


Figure 4: Example of Variance

In the above figure, we can see that our model has learned extremely well for our training data, which has taught it to identify cats. But when given new data, such as the picture of a fox, our model predicts it as a cat, as that is what it has learned. **This happens when the Variance is high, our model will capture all the features of the data given to it, including the noise, will tune itself to the data, and predict it very well but when given new data, it cannot predict on it as it is too specific to training data.**

Hence, our model will perform really well on testing data and get high accuracy but will fail to perform on new, unseen data. New data may not have the exact same features and the model won't be able to predict it very well. This is called Overfitting.

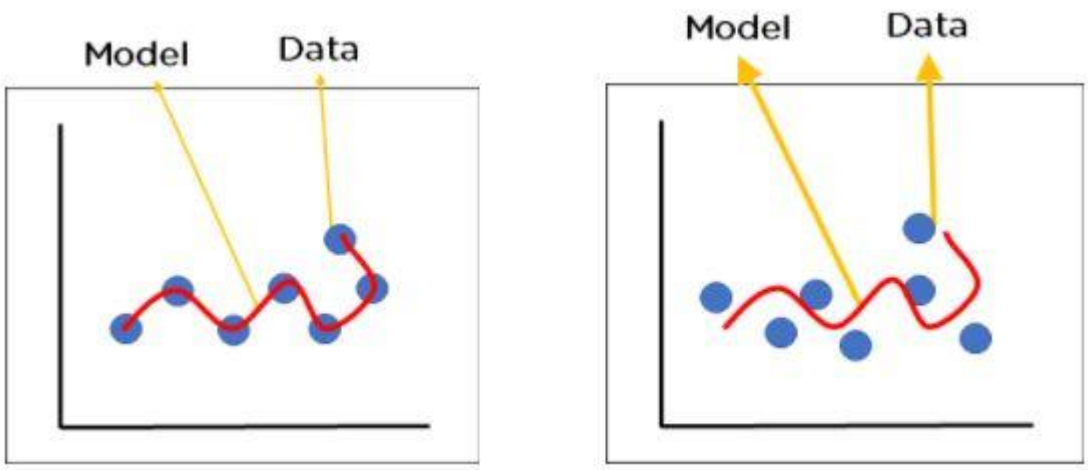


Figure 5: Over-fitted model where we see model performance on, a) training data b) new data

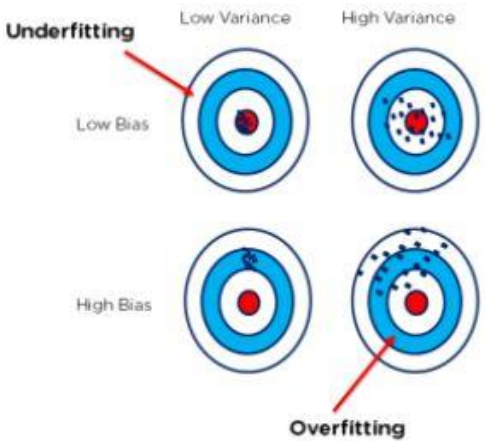


Figure 6: Bull's Eye Graph for Bias and Variance

The above bull's eye graph helps explain bias and variance tradeoff better. The best fit is when the data is concentrated in the center, ie: at the bull's eye. We can see that as we get farther and farther away from the center, the error increases in our model. The best model is one where bias and variance are both low.

Cross-Validation

Cross validation is a technique used in machine learning to evaluate the performance of a model on unseen data. It involves dividing the available data into multiple folds or subsets, using one of these folds as a validation set, and training the model on the remaining folds. This process is repeated multiple times, each time using a different fold as the validation set. Finally, the results from each validation step are averaged to produce a more robust estimate of the model's performance. Cross validation is an important step in the machine learning process and helps to ensure that the model selected for deployment is robust and generalizes well to new data.

cross-validation used for:

The main purpose of cross validation is to prevent overfitting, which occurs when a model is trained too well on the training data and performs poorly on new, unseen data. By evaluating the model on multiple validation sets, cross validation provides a more realistic estimate of the model's generalization performance, i.e., its ability to perform well on new, unseen data.

Types of Cross validation

1. Holdout Validation

In Holdout Validation we perform training on the 50% of the given dataset and rest 50% is used for the testing purpose. It's a simple and quick way to evaluate a model. The major drawback of this method is that we perform training on the 50% of the dataset, it may possible that the remaining 50% of the data contains some important information which we are leaving while training our model that can lead to higher bias.



2. LOOCV (Leave One Out Cross Validation)

In this method we perform training on the whole dataset but leaves only one data-point of the available dataset and then iterates for each data-point. In LOOCV the model is trained on $n-1$ samples and tested on the one omitted sample repeating this process for each data point in the dataset. It has some advantages as well as disadvantages also.

- An advantage of using this method is that we make use of all data points and hence it is low bias.
- The major drawback of this method is that it leads to higher variation in the testing model as we are testing against one data point. If the data point is an outlier it can lead to higher variation.
- Another drawback is it takes a lot of execution time as it iterates over the number of data points we have.

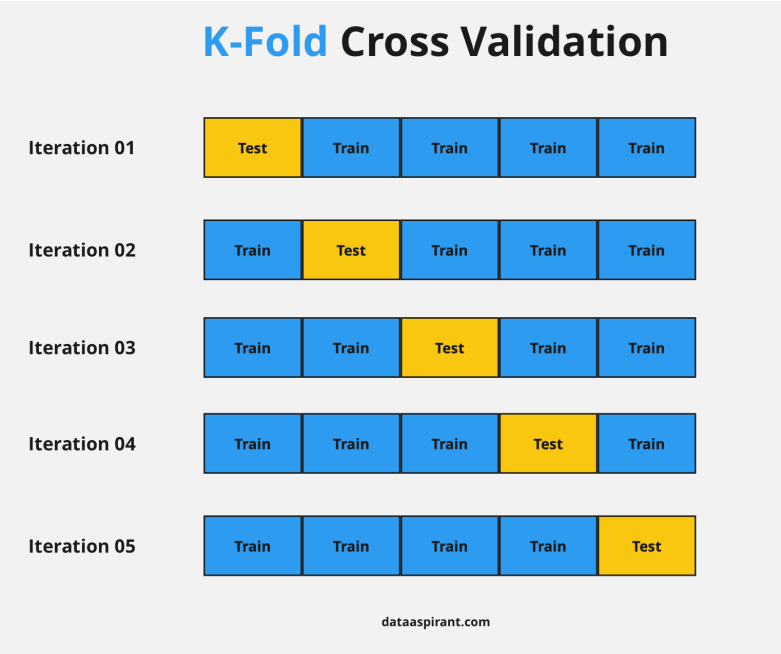


K-Fold Cross Validation

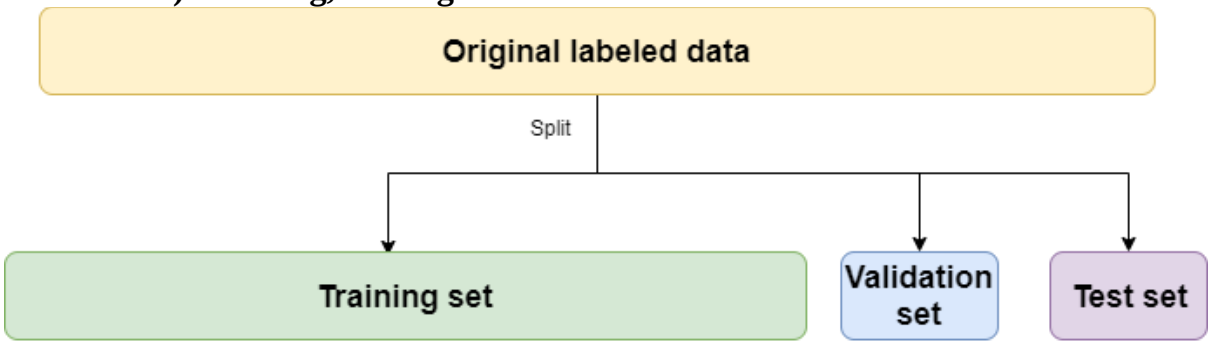
K-Fold Cross Validation is a method used to evaluate a machine learning model by splitting the dataset into K equal parts. The model is trained on K-1 parts and tested on the remaining part, repeating this process K times. The final performance is the average of all test results, offering a more reliable and unbiased estimate of the model's effectiveness.

Working of K-Fold Cross Validation

1. Decide the number of folds, K.
2. Randomly divide the dataset into K equal parts (folds).
3. Select the first fold as the test set and use the remaining K-1 folds to train the model.
4. Test the model on the selected test fold and record the result.
5. Repeat the process K times, each time using a different fold as the test set.
6. Calculate the average of all K results to get the final performance score.



A) Training, Testing and Validation sets



Training Set

The training set is used to fit or train the model. These data points are used to learn the parameters of the model. This is the biggest of all sets in terms of size. The training set includes the features and well as labels in the case of supervised learning. In the case of unsupervised learning, it can simply be the feature sets. These labels are used in the training phase to get the training accuracy score. The training set is usually taken as 70% of the original dataset but can be changed per the use case or available data.

Example

Let's consider a dataset containing 20 points

Dataset1 = [1,5,6,7,8,6,4,5,6,7,23,45,12,34,45,1,7,7,8,0]

Train set can be taken as 60 % of the original Dataset1

The train set will contain 12 data points [8,6,4,5,6,7,23,45,12,34,1,5]

Validation Set

The validation set is used to provide an unbiased evaluation of the model fit during hyperparameter tuning of the model. It is the set of examples that are used to change learning process parameters. Optimal values of hyperparameters are tested against the model trained using the training set. **In Machine Learning or Deep Learning, we generally need to test multiple models with different hyperparameters and check which model gives the best result. This process is carried out with the help of a validation set.**

Example

Let's consider a dataset containing 20 points

Dataset2 = [1,5,6,7,8,6,4,5,6,7,23,45,12,34,45,1,7,7,8,0]

The validation set can be taken as 20 % of the original Dataset2.

The validation set will contain 4 data points [45,1,7,7]

Testing Set

Once we have the model trained with the training set and the hyperparameter tuned using the validation set, we need to test whether the model can generalize well on unseen data. To accomplish this, a test set is used. Here we can check and compare the training and test accuracies. To ensure that the model is not overfitting or underfitting, test accuracies are highly useful. If there is a large difference in train and test accuracies, overfitting might have occurred.

While choosing the test set the below points should be kept in mind:

- The test should contain the same characteristics as of the train set.
- It should be large enough to yield statistically significant results

Example

Let's consider a data set containing 20 points

Dataset3 = [1,5,6,7,8,6,4,5,6,7,23,45,12,34,45,1,7,7,8,0]

The test set can be taken as 20 % of the original Dataset2

The test set will contain 4 data points [6,7,8,0]

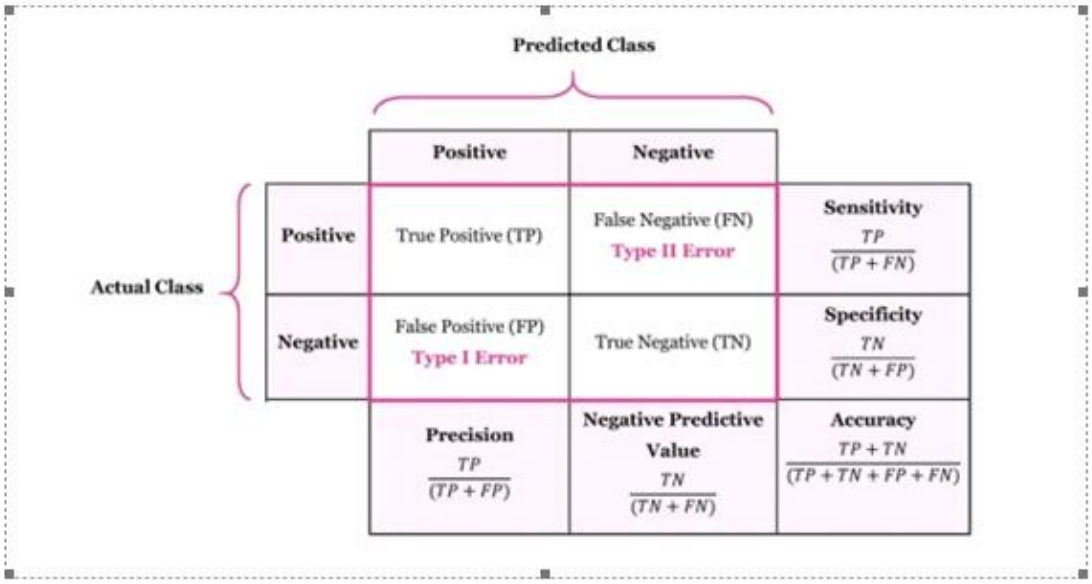
Do we need a train, validation, and test sets?

The training set is necessary to train the model and learn the parameters. Almost all Machine learning/Deep Learning tasks should contain at least a training set.

The validation set and test sets are optional but highly recommended to use because only then can a trained model's legibility and accuracy can be verified. **The validation set can be omitted if we do not choose to perform hyperparameter tuning or model selection**. In such cases, a train set and test set will do the job.

Confusion matrix

A **Confusion matrix** is an ***N x N matrix*** used for evaluating the **performance of a classification model**, where **N** is the number of ***target classes***. The matrix compares the actual target values with those predicted by the machine learning model.



Example:

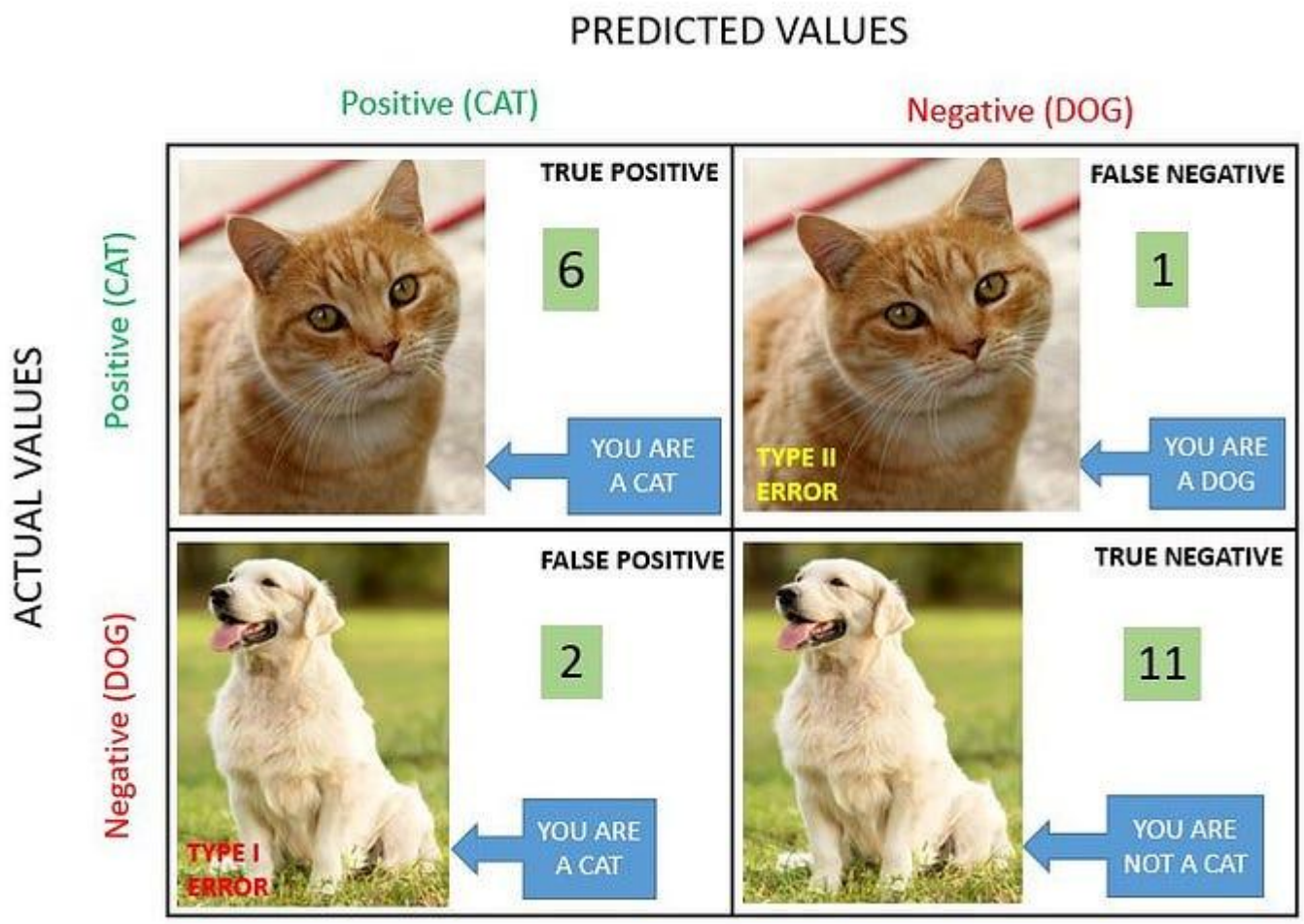
		Predicted	
		Has Cancer	Doesn't Have Cancer
Ground Truth	Has Cancer	TP	FN
	Doesn't Have Cancer	FP	TN

1. **True Positives (TP)**: when the actual value is Positive and predicted is also Positive.
2. **True negatives (TN)**: when the actual value is Negative and prediction is also Negative.
3. **False positives (FP)**: When the actual is negative but prediction is Positive. Also known as the **Type 1 error**
4. **False negatives (FN)**: When the actual is Positive but the prediction is Negative. Also known as the **Type 2 error**

For a binary classification problem, we would have a 2 x 2 matrix as shown above.
example:
We have a total of 2**0 cats and dogs** and our model predicts whether it is a cat or not.

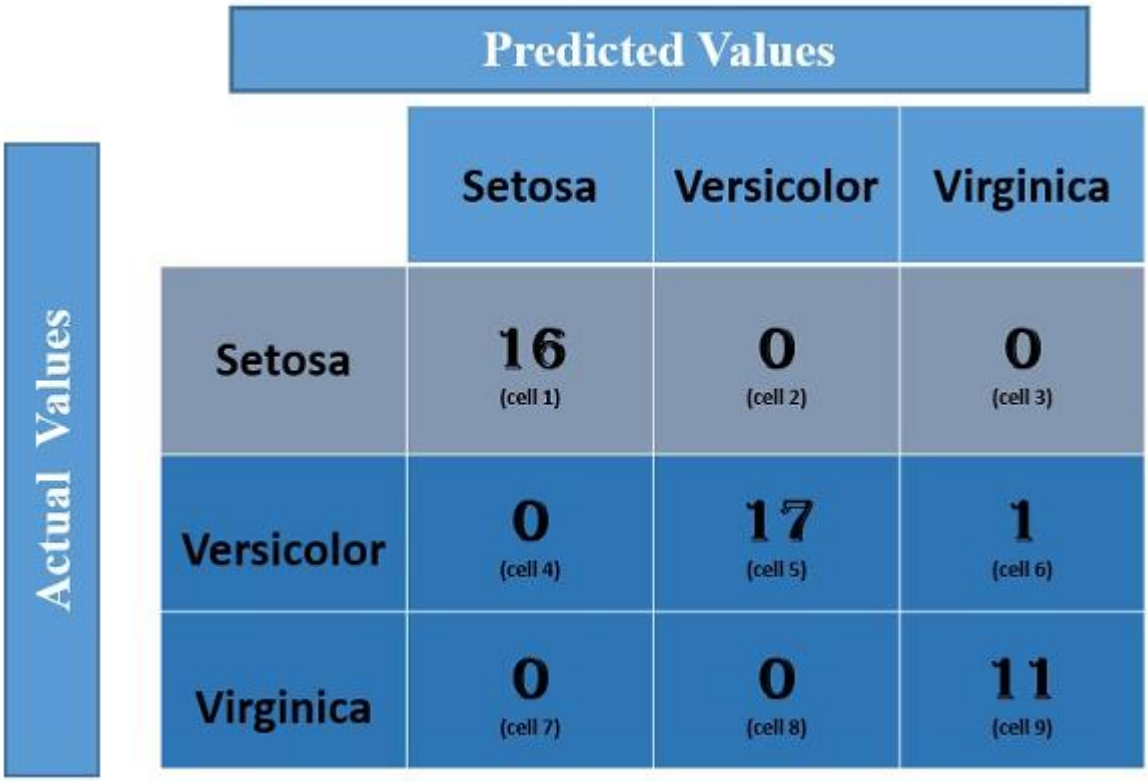
Predicted values =, 'cat', 'cat', 'cat', 'dog', 'dog', 'dog', 'cat', 'dog', 'dog', 'cat', 'dog', 'dog', 'cat']

Actual values	Predicted values
Dog	Dog TN
Cat	Dog FN
Dog	Dog TN
Cat	Cat TP
Dog	Dog TN
Dog	Dog TN
Cat	Cat TP
Dog	Cat FP
Cat	Cat TP
Dog	Cat FP
Dog	Dog TN
Dog	Dog TN
Dog	Dog TN
Cat	Cat TP
Dog	Dog TN
Dog	Dog TN
Cat	Cat TP
Dog	Dog TN
Dog	Dog TN
Cat	Cat TP



True Positive (TP) = 6
You predicted positive and it's true. You predicted that an animal is a cat and it actually is.
True Negative (TN) = 11
You predicted negative and it's true. You predicted that animal is not a cat and it actually is not (it's a dog).
False Positive (Type 1 Error) (FP) = 2
You predicted positive and it's false. You predicted that animal is a cat but it actually is not (it's a dog).
False Negative (Type 2 Error) (FN) = 1
You predicted negative and it's false. You predicted that animal is not a cat but it actually is.

The 3×3 confusion matrix multiclass for the IRIS dataset is as below:



Let us calculate the TP, TN, FP, and FN values for the class **Setosa** using the Above tricks:

TP: The actual value and predicted value should be the same. So concerning Setosa class, the value of cell 1 is the TP value.

FN: The sum of values of corresponding rows except for the TP value

FN = (cell 2 + cell3)

= (0 + 0)

= 0

FP: The sum of values of the corresponding column except for the TP value.

FP = (cell 4 + cell 7)

= (0 + 0)

= 0

TN: The sum of values of all columns and rows except the values of that class that we are calculating the values for.

TN = (cell 5 + cell 6 + cell 8 + cell 9)

= 17 + 1 +0 + 11

= 29

Similarly, for the **Versicolor** class, the values/metrics are calculated as below:

TP: 17 (cell 5)

FN : 0 + 1 = 1 (cell 4 +cell 6)

FP : 0 + 0 = 0 (cell 2 + cell 8)

TN : 16 +0 +0 + 11 =27 (cell 1 + cell 3 + cell 7 + cell 9).

You can try for the Virginia class.

There are measures other than the confusion matrix which can help achieve better understanding and analysis of our model and its performance.

a. **Accuracy:** number of correct predictions divided by the total number of predictions

$$\text{Accuracy} = \frac{\text{Number of correct predictions}}{\text{Total number of predictions}}$$

It can be formulated as:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

Let's try calculating accuracy for the following model that classified 100 tumors as malignant (the positive class) or benign (the negative class):

True Positive (TP):

- Reality: Malignant
- ML model predicted: Malignant
- Number of TP results: 1

False Positive (FP):

- Reality: Benign
- ML model predicted: Malignant
- Number of FP results: 1

False Negative (FN):

- Reality: Malignant
- ML model predicted: Benign
- Number of FN results: 8

True Negative (TN):

- Reality: Benign
- ML model predicted: Benign
- Number of TN results: 90

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} = \frac{1 + 90}{1 + 90 + 1 + 8} = 0.91$$

Accuracy comes out to 0.91, or 91% (91 correct predictions out of 100 total examples). That means our tumor classifier is doing a great job of identifying malignancies
Note: It is not suited for imbalanced dataset.

b. **Precision**: **The precision determines the proportion of positive prediction that was actually correct.** It can be calculated as the True Positive or predictions that are actually true to the total positive predictions (True Positive and False Positive).

$$\textbf{\textit{Precision}} = \frac{\textbf{\textit{TP}}}{(\textbf{\textit{TP}} + \textbf{\textit{FP}})}$$

Let's calculate precision for our ML model from the previous section that analyzes tumors:

True Positives (TPs): 1	False Positives (FPs): 1
False Negatives (FNs): 8	True Negatives (TNs): 90

Precision=$TP/(TP+FP)$=1/(1+1)=0.5
Our model has a precision of 0.5—in other words, when it predicts a tumor is malignant, it is correct 50% of the time.

c. Recall (TPR, Sensitivity):

It calculate the proportion of actual positive that was identified incorrectly. It can be calculated as True Positive or predictions that are actually true to the total number of positives, either correctly predicted as positive or incorrectly predicted as negative (true Positive and false negative).
Let's calculate recall for our tumor classifier:

True Positives (TPs): 1	False Positives (FPs): 1
False Negatives (FNs): 8	True Negatives (TNs): 90

Recall=$TP/(TP+FN)$=1/(1+8)=0.11
Our model has a recall of 0.11—in other words, it correctly identifies 11% of all malignant tumors.

d. Specificity

Specificity measures the proportion of actual negative instances that are correctly identified by the model as negative. It represents the ability of the model to correctly identify negative instances

$$Specificity = \frac{TN}{TN + FP}$$

=90/90+1=0.9

e. F1-Score

F-score or F1 Score is a metric to evaluate a binary classification model on the basis of predictions that are made for the positive class. It is calculated with the help of Precision and Recall. It is a type of single score that represents both Precision and Recall. So, ***the F1 Score can be calculated as the harmonic mean of both precision and Recall, assigning equal weight to each of them.***

The formula for calculating the F1 score is given below:

$$F1 - score = 2 * \frac{precision * recall}{precision + recall}$$

Use F1 Score When:

- You want a **balanced view** between false positives and false negatives
- You're working with **imbalanced datasets**

f. **AUC-ROC** Receiver Operating Characteristic

ROC represents a graph to show the performance of a classification model at different threshold levels. The curve is plotted between two parameters, which are:

- **True Positive Rate**
- **False Positive Rate**

TPR or true Positive rate is a synonym for Recall, hence can be calculated as:

$$TPR = \frac{TP}{TP + FN}$$

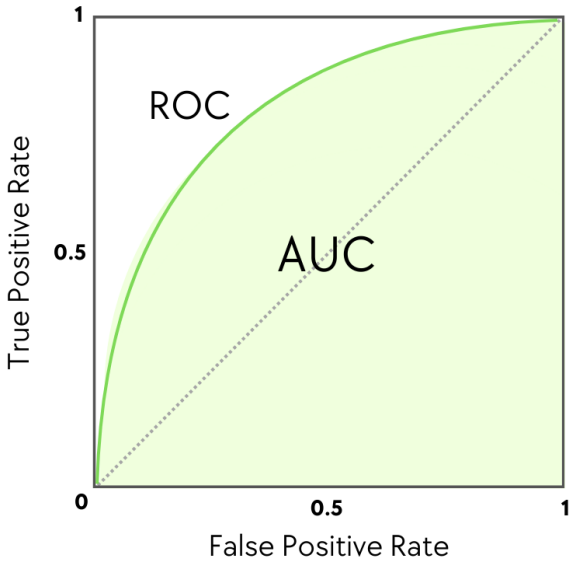
FPR or False Positive Rate can be calculated as:

$$FPR = \frac{FP}{FP + TN}$$

To calculate value at any point in a ROC curve, we can evaluate a logistic regression model multiple times with different classification thresholds, but this would not be much efficient. So, for this, one efficient method is used, which is known as AUC.

AUC: Area Under the ROC curve

AUC is known for **Area Under the ROC curve**. As its name suggests, AUC calculates the two-dimensional area under the entire ROC curve, as shown below image:



AUC calculates the performance across all the thresholds and provides an aggregate measure. The value of AUC ranges from 0 to 1. It means a model with 100% wrong prediction will have an AUC of 0.0, whereas models with 100% correct predictions will have an AUC of 1.0.

Note: where the dataset is highly imbalanced, **the ROC curve can give an overly optimistic assessment of the model's performance**. This optimism bias arises because the ROC curve's false positive rate (FPR) can become very small when the number of actual negatives is large.

Looking at the FPR formula,

$$\text{False Posivite Rate (FPR)} = \frac{FP}{FP+TN}$$

In an ROC curve, the x-axis typically represents the False Positive Rate (FPR), and the y-axis represents the True Positive Rate (TPR), also known as Sensitivity or Recall. So, a higher x-axis value (towards the right) on the ROC curve does indicate a higher False Positive Rate, and a higher y-axis value (towards the top) indicates a higher True Positive Rate.The ROC curve is a graphical representation of the trade-off between true positive rate and false positive rate at various thresholds.
It shows the performance of a classification model at different classification thresholds. The AUC (Area Under the Curve) is a summary measure of the ROC curve performance.The choice of the threshold depends on the specific requirements of the problem you're trying to solve and the trade-off between false positives and false negatives that is acceptable in your context.

- **If you want to prioritize reducing false positives (minimizing the chances of labeling something as positive when it's not), you might choose a threshold that results in a lower false positive rate.**
- **If you want to prioritize increasing true positives (capturing as many actual positives as possible), you might choose a threshold that results in a higher true positive rate.**

Let's consider an example to illustrate how ROC curves are generated for different thresholds and how a particular threshold corresponds to a confusion matrix. Suppose we have a binary classification problem with a model predicting whether an email is spam (positive) or not spam (negative).
Let us consider the hypothetical data,
True Labels: [1, 0, 1, 0, 1, 1, 0, 0, 1, 0]
Predicted Probabilities: [0.8, 0.3, 0.6, 0.2, 0.7, 0.9, 0.4, 0.1, 0.75, 0.55]
Case 1: Threshold = 0.5

True Labels	Predicted Probabilities	Predicted Labels (if Threshold = 0.5)
1	0.8	1
0	0.3	0
1	0.6	1
0	0.2	0
1	0.7	1

True Labels	Predicted Probabilities	Predicted Labels (if Threshold = 0.5)
1	0.9	1
0	0.4	0
0	0.1	0
1	0.75	1
0	0.55	1

Confusion matrix based on above predictions

	Prediction = 0	Prediction = 1
Actual = 0	TP=4	FN=1
Actual = 1	FP=0	TN=5

Accordingly,

- **True Positive Rate (TPR)**:
  Proportion of actual positives correctly identified by the classifier is

$$\text{True Positive Rate (TPR)} = \frac{TP}{TP+FN}$$

  TPR=4/(4+1)=0.8
- **False Positive Rate (FPR)**:
  Proportion of actual negatives incorrectly classified as positives

$$\text{False Posivite Rate (FPR)} = \frac{FP}{FP+TN}$$

  FPR=0/(0+5)= 0

So, at the threshold of 0.5:

- True Positive Rate (Sensitivity): 0.8
- False Positive Rate: 0

The interpretation is that the model, at this threshold, correctly identifies 80% of actual positives (TPR) but incorrectly classifies 0% of actual negatives as positives (FPR).
Accordingly for different thresholds we will get ,

Case 2: Threshold = 0.7

True Labels	Predicted Probabilities	Predicted Labels (if Threshold = 0.7)
1	0.8	1
0	0.3	0
1	0.6	0
0	0.2	0
1	0.7	0
1	0.9	1
0	0.4	0
0	0.1	0
1	0.75	1
0	0.55	0

Confusion matrix based on above predictions

	Prediction = 0	Prediction = 1
Actual = 0	TP=5	FN=0
Actual = 1	FP=2	TN=3

Accordingly,

- **True Positive Rate (TPR)**:
  Proportion of actual positives correctly identified by the classifier is

$$\text{True Positive Rate (TPR)} = \frac{TP}{TP+FN}$$

- **False Positive Rate (FPR)**:
  Proportion of actual negatives incorrectly classified as positives

$$\text{False Posivite Rate (FPR)} = \frac{FP}{FP+TN}$$

Case 3: Threshold = 0.4

True Labels	Predicted Probabilities	Predicted Labels (if Threshold = 0.4)
1	0.8	1
0	0.3	0
1	0.6	1
0	0.2	0
1	0.7	1
1	0.9	1
0	0.4	0
0	0.1	0
1	0.75	1
0	0.55	1

Confusion matrix based on above predictions

	Prediction = 0	Prediction = 1
Actual = 0	TP=4	FN=1
Actual = 1	FP=0	TN=5

Accordingly,

- **True Positive Rate (TPR)**:
  Proportion of actual positives correctly identified by the classifier is

$$\text{True Positive Rate (TPR)} = \frac{TP}{TP+FN}$$

- **False Positive Rate (FPR)**:
  Proportion of actual negatives incorrectly classified as positives

$$\text{False Posivite Rate (FPR)} = \frac{FP}{FP+TN}$$

Case 4: Threshold = 0.2

True Labels	Predicted Probabilities	Predicted Labels (if Threshold = 0.2)
1	0.8	1
0	0.3	1
1	0.6	1
0	0.2	0
1	0.7	1
1	0.9	1
0	0.4	1
0	0.1	0
1	0.75	1
0	0.55	1

Confusion matrix based on above predictions

	Prediction = 0	Prediction = 1

	Prediction = 0	Prediction = 1
Actual = 0	TP=2	FN=3
Actual = 1	FP=0	TN=5

Accordingly,

- **True Positive Rate (TPR)**:
  Proportion of actual positives correctly identified by the classifier is

$$\text{True Positive Rate (TPR)} = \frac{TP}{TP+FN}$$

- **False Positive Rate (FPR)**:
  Proportion of actual negatives incorrectly classified as positives

$$\text{False Posivite Rate (FPR)} = \frac{FP}{FP+TN}$$

Case 5: Threshold = 0.85

True Labels	Predicted Probabilities	Predicted Labels (if Threshold = 0.85)
1	0.8	0
0	0.3	0
1	0.6	0
0	0.2	0
1	0.7	0
1	0.9	1
0	0.4	0
0	0.1	0

True Labels	Predicted Probabilities	Predicted Labels (if Threshold = 0.85)
1	0.75	0
0	0.55	0

Confusion matrix based on above predictions

	Prediction = 0	Prediction = 1
Actual = 0	TP=5	FN=0
Actual = 1	FP=4	TN=1

Accordingly,

- **True Positive Rate (TPR)**:
  Proportion of actual positives correctly identified by the classifier is

$$\text{False Posivite Rate (FPR)} = \frac{FP}{FP+TN}$$

- **False Positive Rate (FPR)**:
  Proportion of actual negatives incorrectly classified as positives

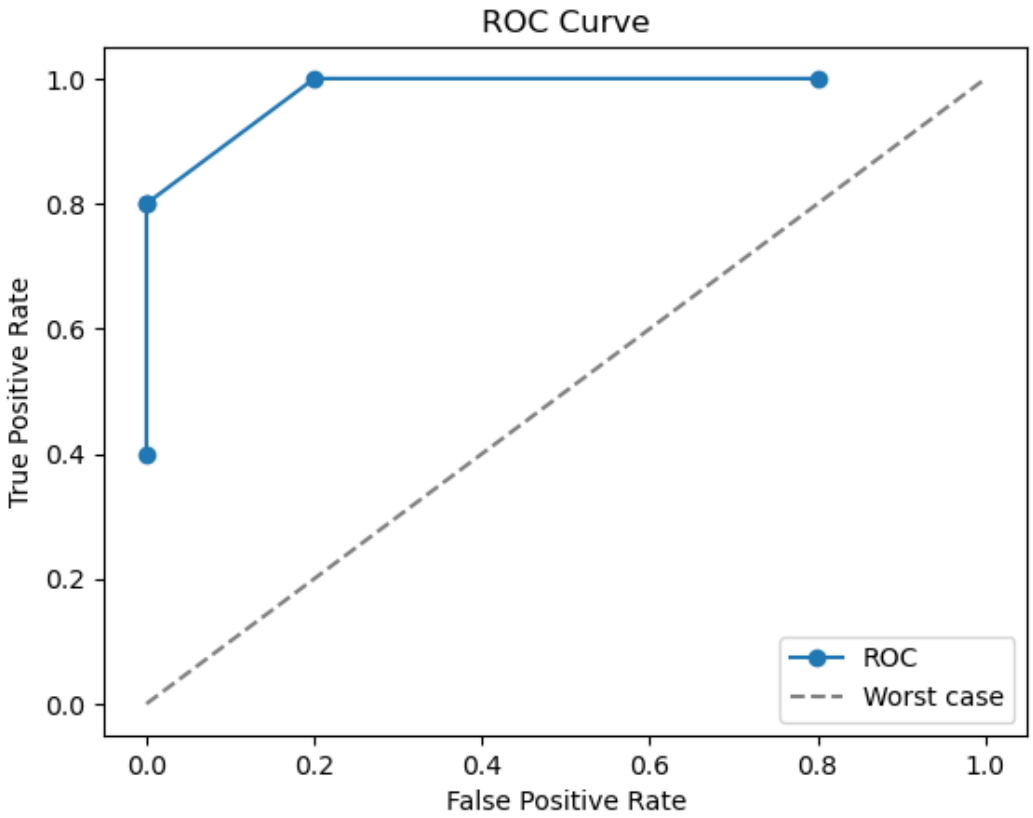
$$\text{False Posivite Rate (FPR)} = \frac{FP}{FP+TN}$$

Based on the above result, we will plot the ROC curve

```
import matplotlib.pyplot as plt
true_positive_rate = [0.4, 0.8,  0.8, 1.0, 1]
false_positive_rate = [0, 0,  0, 0.2, 0.8]

plt.plot(false_positive_rate, true_positive_rate, 'o-', label='ROC')
plt.plot([0, 1], [0, 1], '--', color='grey', label='Worst case')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC Curve')
plt.legend()
plt.show()
```

Output:



From the graph it is implied that:

- The gray dashed line represents the "Worst case" scenario, where the model's predictions i.e TPR are FPR are same. This diagonal line is considered the worst-case scenario, indicating an equal likelihood of false positives and false negatives.
- As points deviate from the random guess line towards the upper-left corner, the model's performance improves.
- The Area Under the Curve (AUC) is a quantitative measure of the model's discriminative ability. **A higher AUC value, closer to 1.0, indicates superior performance. The best possible AUC value is 1.0, corresponding to a model that achieves 100% sensitivity and 100% specificity.**
- In all, the Receiver Operating Characteristic (ROC) curve serves as a graphical representation of the trade-off between a binary classification model's True Positive Rate (sensitivity) and False Positive Rate at various decision thresholds. As the curve gracefully ascends towards the upper-left corner, it signifies the model's commendable ability to discriminate between positive and negative instances across a range of confidence thresholds. This upward trajectory indicates an improved performance, with higher sensitivity achieved while minimizing false positives. The annotated thresholds, denoted as A, B, C, D, and E, offer valuable insights into the model's dynamic behavior at different confidence levels.

$$TPR = \frac{TP}{Actual\ Positive} = \frac{TP}{TP + FN}$$

$$FNR = \frac{FN}{Actual\ Positive} = \frac{FN}{TP + FN}$$

$$TNR = \frac{TN}{Actual\ Negative} = \frac{TN}{TN + FP}$$

$$FPR = \frac{FP}{Actual\ Negative} = \frac{FP}{TN + FP}$$

Even if data is imbalanced, we can figure out that our model is working well or not. For that, **the values of TPR and TNR should be high, and FPR and FNR should be as low as possible.**

Turning Data into Probabilities

Probability simply talks about how **likely is the event to occur,** and its value always lies between **0 and 1** (inclusive of 0 and 1).

- **For example:** consider that you have two bags, named A and B, each containing 10 red balls and 10 black balls. If you randomly pick up the ball from any bag (without looking in the bag), you surely don't know which ball you're going to pick up. So here is the need of probability where we find how likely you're going to pick up either a black or a red ball.

- Note that we'll be denoting probability as P from now on. P(X) means the probability for an event X to occur.

Joint Probability

Joint Probability Probability of events A and B denoted by P(A and B) or P(A ∩ B) is the probability **that events A and B both occur**.

P(A ∩ B) = P(A). P(B)

This only applies if A and B are independent, which means that if A occurred, that doesn't change the probability of B, and vice versa.

For example, suppose a survey asks people about their **favourite color and animal**. The researchers organize the results in the table below:

	Cat	Dog	Other
Red	0.10	0.05	0.03
Green	0.08	0.12	0.05
Blue	0.03	0.06	0.02

We want to find the **likelihood that someone chooses red as their favorite color and a dog as their favourite animal**. We can locate the intersection of the "Red" row and the "Dog" column, which is 0.05.

Therefore, the joint probability is:
$P(\text{Red} \cap \text{Dog}) = 0.05$

For example, suppose we have a coin that we flip twice. We want to find the chances of getting heads on both the first and second flips. Because each flip is independent, the probability of the first heads is 1/2, and the likelihood of heads on the second flip is also 1/2.
Therefore, the joint probability is the following:
$P(H1 \cap H2)$ = P(heads on first flip) x P(tails on second flip)
= 1/2 x 1/2
= 1/4

Joint Probability Formula for Dependent Events

We can use the general multiplication rule to calculate joint probabilities for dependent events. This rule allows us to factor in **how the occurrence of one event affects the likelihood of the other event.**
The joint probability formula for dependent events is the following:
$P(A \cap B) = P(A) * P(B|A)$
Here, **P(A) represents the chances of event A occurring, while P(B|A) represents the conditional probability of event B occurring, given that event A has already happened.** By multiplying these two likelihoods, we can calculate the joint probability of both events coinciding.

Example:
Health Risk Assessment Scenario:

- **Based on statistical data, let's assume that the probability of a randomly selected person being a smoker is 0.2**
- **and the probability of a randomly selected person being obese is 0.3.**
- **Studies have shown that among the smokers, 0.1 are obese.**
- **What's the joint probability that a randomly selected person is both a smoker and obese?**

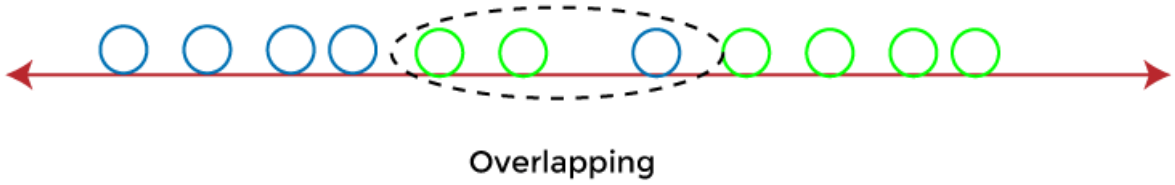
Solution:
Here, the events are dependent.
The joint probability would be
$P(\text{Smoker and Obese}) = P(\text{Smoker}) \times P(\text{Obese}|\text{Smoker})$
$= 0.2 \times 0.1$
$= 0.02$ or 2%.

Dimensionality Reduction:

LDA

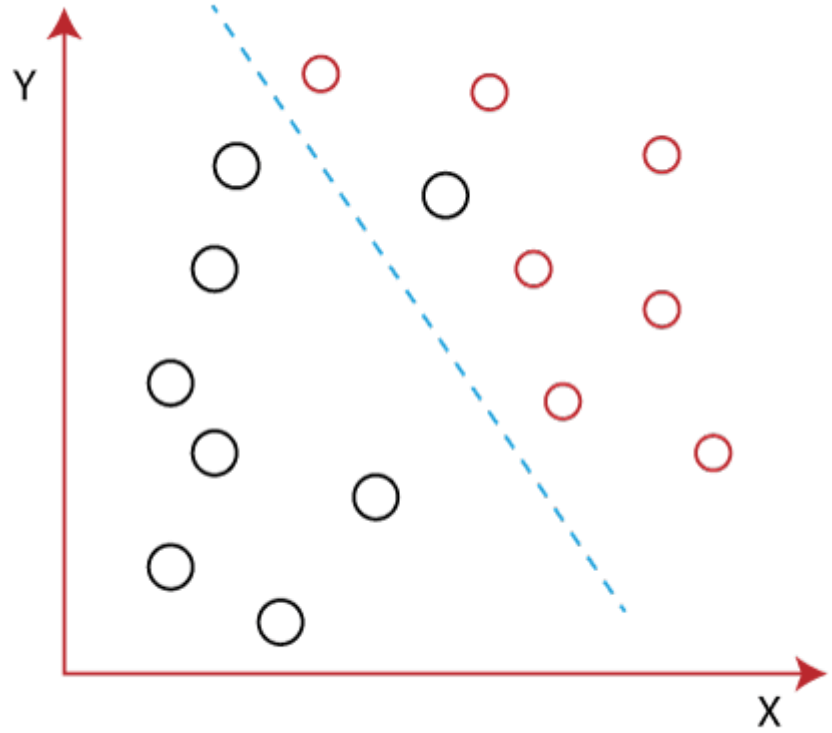
Linear Discriminant analysis ***is one of the most popular dimensionality reduction techniques used for supervised classification problems in machine learning***. It is also considered a pre-processing step for modelling differences in ML and applications of pattern classification.

Whenever there is a requirement to separate two or more classes having multiple features efficiently, the Linear Discriminant Analysis model is considered the most common technique to solve such classification problems. For e.g., if we have two classes with multiple features and need to separate them efficiently. When we classify them using a single feature, then it may show overlapping.



Overlapping

To overcome the overlapping issue in the classification process, we must increase the number of features regularly.
Example:
Let's assume we have to classify two different classes having two sets of data points in a 2-dimensional plane as shown below image:



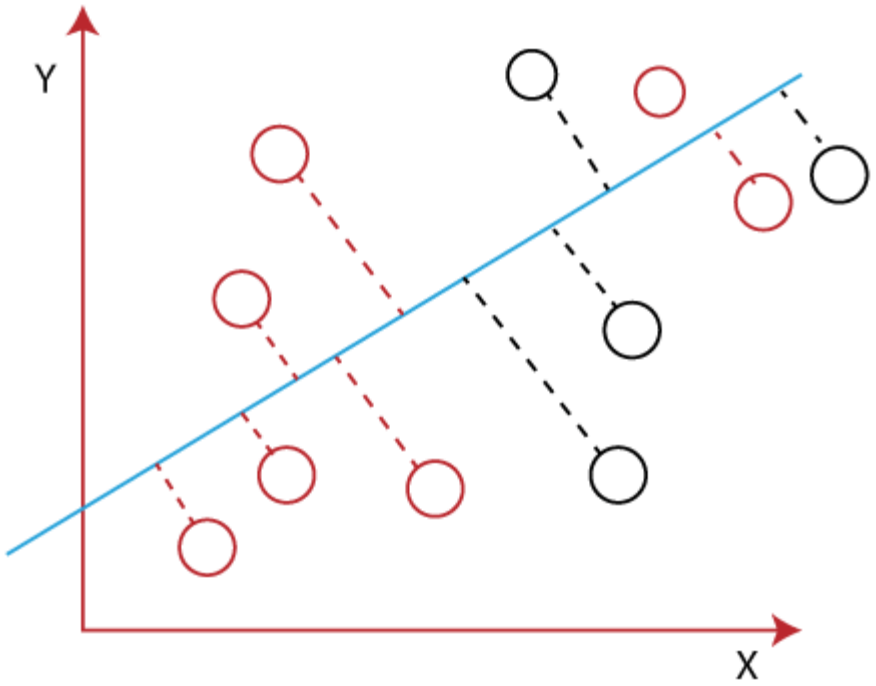
However, it is impossible to draw a straight line in a 2-d plane that can separate these data points efficiently but using linear Discriminant analysis; we can dimensionally reduce the 2-D plane into the 1-D plane. Using this technique, we can also maximize the separability between multiple classes.

How Linear Discriminant Analysis (LDA) works?

Linear Discriminant analysis is used as a dimensionality reduction technique in machine learning, using which we can easily **transform a 2-D and 3-D graph into a 1-dimensional plane.**

Let's consider an example where we have two classes in a 2-D plane having an X-Y axis, and we need to classify them efficiently. As we have already seen in the above example that LDA enables us to draw a straight line that can completely separate the two classes of the data points. Here, LDA uses an X-Y axis to create a new axis by separating them using a straight line and projecting data onto a new axis.

Hence, we can maximize the separation between these classes and reduce the 2-D plane into 1-D.



To create a new axis, Linear Discriminant Analysis uses the following criteria:

- It maximizes the distance between means of two classes.
- It minimizes the variance within the individual class.

Using the above two conditions, LDA generates a new axis in such a way that it can maximize the distance between the means of the two classes and minimizes the variation within each class.

In other words, we can say that the new axis will increase the separation between the data points of the two classes and plot them onto the new axis.

Why LDA?

- Logistic Regression is one of the most popular classification algorithms that perform well for binary classification but falls short in the case of multiple classification problems with well-separated classes. At the same time, LDA handles these quite efficiently.
- LDA can also be used in data pre-processing to reduce the number of features, just as PCA, which reduces the computing cost significantly.
- LDA is also used in face detection algorithms.

Drawbacks of Linear Discriminant Analysis (LDA)

Although, LDA is specifically used to solve supervised classification problems for two or more classes which are not possible using logistic regression in machine learning. But LDA also fails in some cases where the Mean of the distributions is shared. In this case, LDA fails to create a new axis that makes both the classes linearly separable.

To overcome such problems, we use **non-linear Discriminant analysis** in machine learning.

Real-world Applications of LDA

Some of the common real-world applications of Linear discriminant Analysis are given below:

- **Face Recognition**
  Face recognition is the popular application of computer vision, where each face is represented as the combination of a number of pixel values. In this case, LDA is used to minimize the number of features to a manageable number before going through the classification process. It generates a new template in which each dimension consists of a linear combination of pixel values.
- **Medical**
  In the medical field, LDA has a great application in classifying the patient disease on the basis of various parameters of patient health and the medical treatment which is going on. On such parameters, it classifies disease as mild, moderate, or severe. This classification helps the doctors in either increasing or decreasing the pace of the treatment.

Etc

Linear Discriminant Analysis - Steps

1. Compute the class means of dependent variable

$$\mu_1 = \frac{1}{N_1}\sum_{x\in\omega_1} x$$

2. Derive the covariance matrix of the class variable

$$S_1 = \sum_{x\in\omega_1}(x-\mu_1)(x-\mu_1)^T$$

3. Compute the within class — scatter matrix (S1+S2)

$$S_w = S_1 + S_2$$

4. Compute the between class scatter matrix

$$S_B = (\mu_1-\mu_2)(\mu_1-\mu_2)^T$$

5. Compute the Eigen values and eigen vectors from the within class and between class scatter matrix

$$S_W^{-1}S_B w = \lambda w$$

6. Sort the values of eigen values and select the top k values
7. Find the eigen vectors corresponds to the top k eigen vectors

$$\left(S_W^{-1}S_B - \lambda I\right)\begin{pmatrix} w_1 \\ w_2 \end{pmatrix} = 0$$

8. Obtain the LDA by taking the dot product of eigen vectors and original data

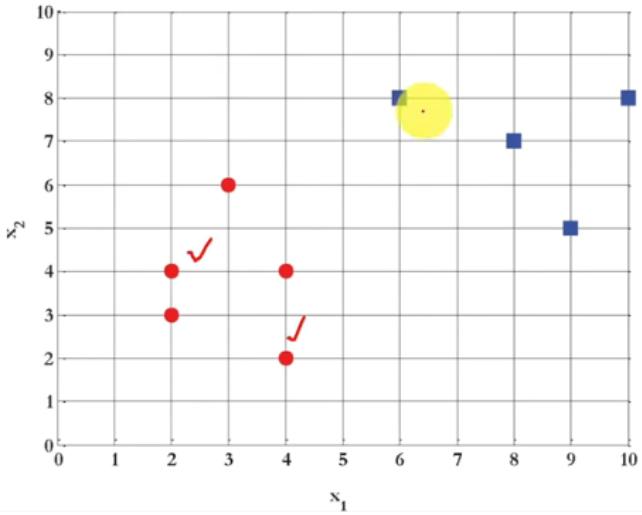
Example:

- Compute the Linear Discriminant projection for the following two dimensional dataset.

Samples for class ω_1 : $\mathbf{X}_1=(x_1,x_2)=\{(4,2),(2,4),(2,3),(3,6),(4,4)\}$

Sample for class ω_2 : $\mathbf{X}_2=(x_1,x_2)=\{(9,10),(6,8),(9,5),(8,7),(10,8)\}$

Dataset is in 2D space now we have to convert to one dimensional space



Step1: Calculate means of both the classes

Samples for class ω_1 : $\mathbf{X}_1=(x_1,x_2)=\{(4,2),(2,4),(2,3),(3,6),(4,4)\}$

Sample for class ω_2 : $\mathbf{X}_2=(x_1,x_2)=\{(9,10),(6,8),(9,5),(8,7),(10,8)\}$

The classes mean are :

$$\mu_1 = \frac{1}{N_1}\sum_{x\in\omega_1} x = \frac{1}{5}\left[\begin{pmatrix}4\\2\end{pmatrix}+\begin{pmatrix}2\\4\end{pmatrix}+\begin{pmatrix}2\\3\end{pmatrix}+\begin{pmatrix}3\\6\end{pmatrix}+\begin{pmatrix}4\\4\end{pmatrix}\right] = \begin{pmatrix}3\\3.8\end{pmatrix}$$

$$\mu_2 = \frac{1}{N_2}\sum_{x\in\omega_2} x = \frac{1}{5}\left[\begin{pmatrix}9\\10\end{pmatrix}+\begin{pmatrix}6\\8\end{pmatrix}+\begin{pmatrix}9\\5\end{pmatrix}+\begin{pmatrix}8\\7\end{pmatrix}+\begin{pmatrix}10\\8\end{pmatrix}\right] = \begin{pmatrix}8.4\\7.6\end{pmatrix}$$

Step2: Calculate the Covariance matrix of two classes:

Samples for class ω_1 : $\mathbf{X_1}=(x_1,x_2)=\{(4,2),(2,4),(2,3),(3,6),(4,4)\}$

Sample for class ω_2 : $\mathbf{X_2}=(x_1,x_2)=\{(9,10),(6,8),(9,5),(8,7),(10,8)\}$

Covariance matrix of the first class:

$$S_1 = \sum_{x\in\omega_1} \frac{(x-\mu_1)(x-\mu_1)^T}{\text{N-1}} = \left[\begin{pmatrix}4\\2\end{pmatrix}-\begin{pmatrix}3\\3.8\end{pmatrix}\right]\left[\begin{pmatrix}4\\2\end{pmatrix}-\begin{pmatrix}3\\3.8\end{pmatrix}\right]^T+\left[\begin{pmatrix}2\\4\end{pmatrix}-\begin{pmatrix}3\\3.8\end{pmatrix}\right]\left[\begin{pmatrix}2\\4\end{pmatrix}-\begin{pmatrix}3\\3.8\end{pmatrix}\right]^T$$
$$+\left[\begin{pmatrix}2\\3\end{pmatrix}-\begin{pmatrix}3\\3.8\end{pmatrix}\right]\left[\begin{pmatrix}2\\3\end{pmatrix}-\begin{pmatrix}3\\3.8\end{pmatrix}\right]^T+\left[\begin{pmatrix}3\\6\end{pmatrix}-\begin{pmatrix}3\\3.8\end{pmatrix}\right]\left[\begin{pmatrix}3\\6\end{pmatrix}-\begin{pmatrix}3\\3.8\end{pmatrix}\right]^T+\left[\begin{pmatrix}4\\4\end{pmatrix}-\begin{pmatrix}3\\3.8\end{pmatrix}\right]\left[\begin{pmatrix}4\\4\end{pmatrix}-\begin{pmatrix}3\\3.8\end{pmatrix}\right]^T \Big/ \text{N-1}$$
$$=\begin{pmatrix}1 & -0.25\\ -0.25 & 2.2\end{pmatrix}$$

Covariance matrix of the second class:

$$S_2 = \sum_{x\in\omega_2} \frac{(x-\mu_2)(x-\mu_2)^T}{\text{N-1}} = \left[\begin{pmatrix}9\\10\end{pmatrix}-\begin{pmatrix}8.4\\7.6\end{pmatrix}\right]\left[\begin{pmatrix}9\\10\end{pmatrix}-\begin{pmatrix}8.4\\7.6\end{pmatrix}\right]^T+\left[\begin{pmatrix}6\\8\end{pmatrix}-\begin{pmatrix}8.4\\7.6\end{pmatrix}\right]\left[\begin{pmatrix}6\\8\end{pmatrix}-\begin{pmatrix}8.4\\7.6\end{pmatrix}\right]^T$$
$$+\left[\begin{pmatrix}9\\5\end{pmatrix}-\begin{pmatrix}8.4\\7.6\end{pmatrix}\right]\left[\begin{pmatrix}9\\5\end{pmatrix}-\begin{pmatrix}8.4\\7.6\end{pmatrix}\right]^T+\left[\begin{pmatrix}8\\7\end{pmatrix}-\begin{pmatrix}8.4\\7.6\end{pmatrix}\right]\left[\begin{pmatrix}8\\7\end{pmatrix}-\begin{pmatrix}8.4\\7.6\end{pmatrix}\right]^T+\left[\begin{pmatrix}10\\8\end{pmatrix}-\begin{pmatrix}8.4\\7.6\end{pmatrix}\right]\left[\begin{pmatrix}10\\8\end{pmatrix}-\begin{pmatrix}8.4\\7.6\end{pmatrix}\right]^T \Big/ \text{N-1}$$
$$=\begin{pmatrix}2.3 & -0.05\\ -0.05 & 3.3\end{pmatrix}$$

Step 3: calculate within class scatter matrix

Within-class scatter matrix:

$$S_w = S_1 + S_2 = \begin{pmatrix}1 & -0.25\\ -0.25 & 2.2\end{pmatrix}+\begin{pmatrix}2.3 & -0.05\\ -0.05 & 3.3\end{pmatrix}$$
$$=\begin{pmatrix}3.3 & -0.3\\ -0.3 & 5.5\end{pmatrix}$$

Step 3: calculate between class scatter matrix

$$S_B = (\mu_1 - \mu_2)(\mu_1 - \mu_2)^T$$

$$= \left[\begin{pmatrix} 3 \\ 3.8 \end{pmatrix} - \begin{pmatrix} 8.4 \\ 7.6 \end{pmatrix}\right]\left[\begin{pmatrix} 3 \\ 3.8 \end{pmatrix} - \begin{pmatrix} 8.4 \\ 7.6 \end{pmatrix}\right]^T$$

$$= \begin{pmatrix} -5.4 \\ -3.8 \end{pmatrix}\begin{pmatrix} -5.4 & -3.8 \end{pmatrix}$$

$$= \begin{pmatrix} 29.16 & 20.52 \\ 20.52 & 14.44 \end{pmatrix}$$

Step 5:

Find Eigen Values

$$S_W^{-1}S_B w = \lambda w$$

$$\Rightarrow |S_W^{-1}S_B - \lambda I| = 0$$

$$\Rightarrow \left|\begin{pmatrix} 3.3 & -0.3 \\ -0.3 & 5.5 \end{pmatrix}^{-1}\begin{pmatrix} 29.16 & 20.52 \\ 20.52 & 14.44 \end{pmatrix} - \lambda\begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}\right| = 0$$

$$\Rightarrow \left|\begin{pmatrix} 0.3045 & 0.0166 \\ 0.0166 & 0.1827 \end{pmatrix}\begin{pmatrix} 29.16 & 20.52 \\ 20.52 & 14.44 \end{pmatrix} - \lambda\begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}\right| = 0$$

$$\Rightarrow \begin{vmatrix} 9.2213 - \lambda & 6.489 \\ 4.2339 & 2.9794 - \lambda \end{vmatrix}$$

$$= (9.2213 - \lambda)(2.9794 - \lambda) - 6.489 \times 4.2339 = 0$$

$$\Rightarrow \lambda^2 - 12.2007\lambda = 0 \Rightarrow \lambda(\lambda - 12.2007) = 0$$

$$\Rightarrow \lambda_1 = 0, \lambda_2 = 12.2007$$

Step 6: Find eigen vector

$$w^* = S_W^{-1}(\mu_1 - \mu_2)$$

This is the Linear Discriminant Analysis (LDA) projection vector formula, where:

- S_W is the within-class scatter matrix,
- $\mu 1$ and $\mu 2$ are the mean vectors of the two classes.

$$w^* = S_W^{-1}(\mu_1 - \mu_2) = \begin{pmatrix} 3.3 & -0.3 \\ -0.3 & 5.5 \end{pmatrix}^{-1} \left[\begin{pmatrix} 3 \\ 3.8 \end{pmatrix} - \begin{pmatrix} 8.4 \\ 7.6 \end{pmatrix} \right]$$

$$= \begin{pmatrix} 0.3045 & 0.0166 \\ 0.0166 & 0.1827 \end{pmatrix} \begin{pmatrix} -5.4 \\ -3.8 \end{pmatrix}$$

First element:

0.3045*(−5.4)+0.0166*(−3.8)=−1.6437−0.0631=−1.70680.3045

Second element:

0.0166*(−5.4)+0.1827*(−3.8)=−0.0896−0.6943=−0.7839

So result is:

$$w^* = \begin{pmatrix} -1.7068 \\ -0.7839 \end{pmatrix}$$

Normalize

The **formula for the magnitude** (also called **Euclidean norm** or **length**) of a 2-dimensional vector

$$\mathbf{v} = \begin{pmatrix} x \\ y \end{pmatrix}$$

is:

$$\|\mathbf{v}\| = \sqrt{x^2 + y^2}$$

Let's compute the **magnitude** of this vector:

$$\|w\| = \sqrt{(-1.7068)^2 + (-0.7839)^2} = \sqrt{2.915 + 0.6145} = \sqrt{3.5295} \approx 1.8798$$

Now divide each component by the magnitude:

$$\frac{-1.7068}{1.8798} \approx -0.9088, \quad \frac{-0.7839}{1.8798} \approx -0.4173$$

Take the absolute values so

$$\begin{pmatrix} 0.9088 \\ 0.4173 \end{pmatrix}$$

Step:7 7:

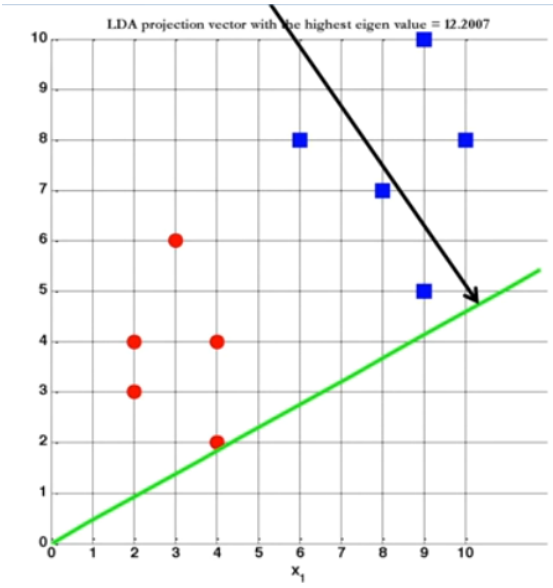
- Obtain the LDA by taking the dot product of eigen vectors and original data

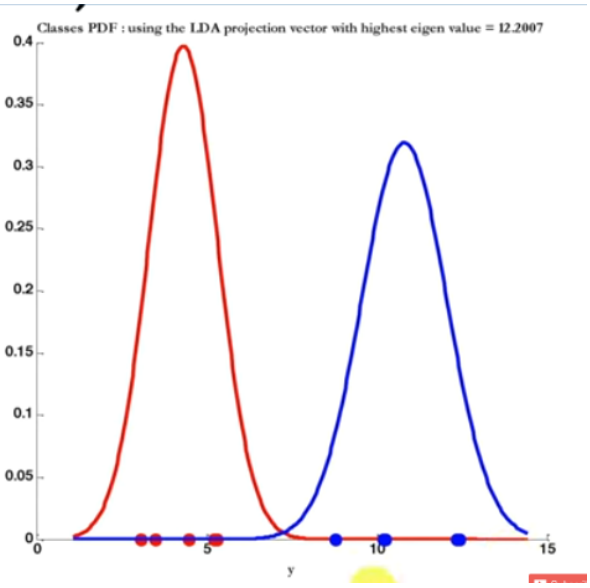
$$w_2 = \begin{pmatrix} 0.9088 \\ 0.4173 \end{pmatrix} = w^*$$

Samples for class ω_1 : $\mathbf{X}_1=(x_1,x_2)=\{(4,2),(2,4),(2,3),(3,6),(4,4)\}$

Sample for class ω_2 : $\mathbf{X}_2=(x_1,x_2)=\{(9,10),(6,8),(9,5),(8,7),(10,8)\}$

X1	4	2	2	3	4	9	6	9	8	10
X2	2	4	3	6	4	10	8	5	7	8
1st LD	4.46	3.48	3.06	5.2	5.3	12.35	8.8	10.2	10.19	12.42





PCA:

Principal Component Analysis is a technique of feature extraction that maps a higher dimensional feature space to a lower-dimensional feature space. While reducing the number of dimensions, PCA ensures that maximum information of the original dataset is retained in the dataset with the reduced no. of dimensions and the co-relation between the newly obtained Principal Components is minimum. The new features obtained after applying PCA are called Principal Components.

let's summarize the approach in a few simple steps:

1. Standardize the *d*-dimensional dataset.
2. Construct the covariance matrix.
3. Decompose the covariance matrix into its eigenvectors and eigenvalues.
4. Sort the eigenvalues by decreasing order to rank the corresponding eigenvectors.
5. Select *k* eigenvectors which correspond to the *k* largest eigenvalues, where *k* is the dimensionality of the new feature subspace ($k \leq d$).

6. Construct a projection matrix ***W*** from the "top" *k* eigenvectors.
7. Transform the *d*-dimensional input dataset ***X*** using the projection matrix ***W*** to obtain the new *k*-dimensional feature subspace.

Example:

- Given the data in Table, reduce the dimension from 2 to 1 using the Principal Component Analysis (PCA) algorithm.

Feature	Example 1	Example 2	Example 3	Example 4
X_1	4	8	13	7
X_2	11	4	5	14

Step 1: Calculate Mean

F	Ex 1	Ex 2	Ex 3	Ex 4
X_1	4	8	13	7
X_2	11	4	5	14

$$\bar{X}_1 = \tfrac{1}{4}(4+8+13+7) = 8$$
$$\bar{X}_2 = \tfrac{1}{4}(11+4+5+14) = 8.5.$$

Step 2: Calculation of the covariance matrix.

F	Ex 1	Ex 2	Ex 3	Ex 4
X_1	4	8	13	7
X_2	11	4	5	14

$\overline{X_1} = 8$

$\overline{X_2} = 8.5$

$$S = \begin{bmatrix} \mathrm{Cov}(X_1, X_1) & \mathrm{Cov}(X_1, X_2) \\ \mathrm{Cov}(X_2, X_1) & \mathrm{Cov}(X_2, X_2) \end{bmatrix}$$

$$\mathrm{Cov}(X_1, X_1) = \frac{1}{N-1}\sum_{k=1}^{N}(X_{1k} - \bar{X}_1)(X_{1k} - \bar{X}_1)$$
$$= \tfrac{1}{3}\left((4-8)^2 + (8-8)^2 + (13-8)^2 + (7-8)^2\right)$$
$$= 14$$

$$\mathrm{Cov}(X_1, X_2) = \frac{1}{N-1}\sum_{k=1}^{N}(X_{1k} - \bar{X}_1)(X_{2k} - \bar{X}_2)$$
$$= \tfrac{1}{3}((4-8)(11-8.5) + (8-8)(4-8.5) + (13-8)(5-8.5) + (7-8)(14-8.5))$$
$$= -11$$

$$\mathrm{Cov}(X_2, X_1) = \mathrm{Cov}(X_1, X_2)$$
$$= -11$$

$$\mathrm{Cov}(X_2, X_2) = \frac{1}{N-1}\sum_{k=1}^{N}(X_{2k} - \bar{X}_2)(X_{2k} - \bar{X}_2)$$
$$= \tfrac{1}{3}\left((11-8.5)^2 + (4-8.5)^2 + (5-8.5)^2 + (14-8.5)^2\right)$$
$$= 23$$

Step 2: Calculation of the covariance matrix.

$$S = \begin{bmatrix} \mathrm{Cov}(X_1, X_1) & \mathrm{Cov}(X_1, X_2) \\ \mathrm{Cov}(X_2, X_1) & \mathrm{Cov}(X_2, X_2) \end{bmatrix}$$

$$= \begin{bmatrix} 14 & -11 \\ -11 & 23 \end{bmatrix}$$

Step 3: Eigenvalues of the covariance matrix

The characteristic equation of the covariance matrix is,

$$0 = \det(S - \lambda I)$$

$$= \begin{vmatrix} 14 - \lambda & -11 \\ -11 & 23 - \lambda \end{vmatrix}$$

$$= (14 - \lambda)(23 - \lambda) - (-11) \times (-11)$$

$$= \lambda^2 - 37\lambda + 201$$

F	Ex 1	Ex 2	Ex 3	Ex 4
X_1	4	8	13	7
X_2	11	4	5	14

$$\overline{X_1} = 8$$

$$\overline{X_2} = 8.5$$

$$S = \begin{bmatrix} 14 & -11 \\ -11 & 23 \end{bmatrix}$$

Step 4: Computation of the eigenvectors

F	Ex 1	Ex 2	Ex 3	Ex 4
X_1	4	8	13	7
X_2	11	4	5	14

$$U = \begin{bmatrix} u_1 \\ u_2 \end{bmatrix} \qquad \begin{bmatrix} 0 \\ 0 \end{bmatrix} = (S - \lambda I)\, U$$

$$= \begin{bmatrix} 14-\lambda & -11 \\ -11 & 23-\lambda \end{bmatrix} \begin{bmatrix} u_1 \\ u_2 \end{bmatrix}$$

$$= \begin{bmatrix} (14-\lambda)u_1 - 11u_2 \\ -11u_1 + (23-\lambda)u_2 \end{bmatrix}$$

$$\overline{X_1} = 8$$

$$\overline{X_2} = 8.5$$

$$S = \begin{bmatrix} 14 & -11 \\ -11 & 23 \end{bmatrix}$$

$$\lambda_1 = 30.3849$$

$$\lambda_2 = 6.6151$$

$(14-\lambda)u_1 = 11u_2$

$$(14-\lambda)u_1 - 11u_2 = 0$$

$$-11u_1 + (23-\lambda)u_2 = 0$$

$$\frac{u_1}{11} = \frac{u_2}{14-\lambda} = t$$

Step 4: Computation of the eigenvectors

$$U = \begin{bmatrix} u_1 \\ u_2 \end{bmatrix}$$

$$\frac{u_1}{11} = \frac{u_2}{14-\lambda} = t$$

$$u_1 = 11t, \quad u_2 = (14-\lambda)t$$

Assume t=1

Step 4: Computation of the eigenvectors

$$U = \begin{bmatrix} u_1 \\ u_2 \end{bmatrix}$$

$$\frac{u_1}{11} = \frac{u_2}{14-\lambda} = t$$

$$u_1 = 11t, \quad u_2 = (14-\lambda)t$$

$$U_{\lambda} = \begin{bmatrix} 11 \\ 14-\lambda \end{bmatrix}.$$

consider largest eigen values .

F	Ex 1	Ex 2	Ex 3	Ex 4
X_1	4	8	13	7
X_2	11	4	5	14

Step 4: Computation of the eigenvectors

$$U_1 = \begin{bmatrix} 11 \\ 14-\lambda_1 \end{bmatrix}.$$

- To find a unit eigenvector, we compute the length of U_1 which is given by,

$$\|U_1\| = \sqrt{11^2 + (14-\lambda_1)^2}$$
$$= \sqrt{11^2 + (14-30.3849)^2}$$
$$= 19.7348$$

$$e_1 = \begin{bmatrix} 11/\|U_1\| \\ (14-\lambda_1)/\|U_1\| \end{bmatrix}$$
$$= \begin{bmatrix} 11/19.7348 \\ (14-30.3849)/19.7348 \end{bmatrix}$$
$$= \begin{bmatrix} 0.5574 \\ -0.8303 \end{bmatrix}$$

$$\overline{X_1} = 8$$

$$\overline{X_2} = 8.5$$

$$S = \begin{bmatrix} 14 & -11 \\ -11 & 23 \end{bmatrix}$$

$$\lambda_1 = 30.3849$$

$$\lambda_2 = 6.6151$$

F	Ex 1	Ex 2	Ex 3	Ex 4
X_1	4	8	13	7
X_2	11	4	5	14

Step 4: Computation of the eigenvectors

$$U_1 = \begin{bmatrix} 11 \\ 14-\lambda_1 \end{bmatrix}.$$

- To find a unit eigenvector, we compute the length of U_1 which is given by,

$$\|U_1\| = \sqrt{11^2 + (14-\lambda_1)^2}$$
$$= \sqrt{11^2 + (14-30.3849)^2}$$
$$= 19.7348$$

$$e_1 = \begin{bmatrix} 11/\|U_1\| \\ (14-\lambda_1)/\|U_1\| \end{bmatrix}$$
$$= \begin{bmatrix} 11/19.7348 \\ (14-30.3849)/19.7348 \end{bmatrix}$$
$$= \begin{bmatrix} 0.5574 \\ -0.8303 \end{bmatrix}$$

$$\overline{X_1} = 8$$

$$\overline{X_2} = 8.5$$

$$S = \begin{bmatrix} 14 & -11 \\ -11 & 23 \end{bmatrix}$$

$$\lambda_1 = 30.3849$$

$$\lambda_2 = 6.6151$$

$$e_2 = \begin{bmatrix} 0.8303 \\ 0.5574 \end{bmatrix}$$

Step 5: Computation of first principal components

$$e_1^T \begin{bmatrix} X_{1k} - \bar{X}_1 \\ X_{2k} - \bar{X}_2 \end{bmatrix}$$

F	Ex 1	Ex 2	Ex 3	Ex 4
X_1	4	8	13	7
X_2	11	4	5	14

$$e_1 = \begin{bmatrix} 0.5574 \\ -0.8303 \end{bmatrix} \qquad \overline{X_1} = 8$$

$$\overline{X_2} = 8.5$$

$$e_2 = \begin{bmatrix} 0.8303 \\ 0.5574 \end{bmatrix} \qquad S = \begin{bmatrix} 14 & -11 \\ -11 & 23 \end{bmatrix}$$

$$\lambda_1 = 30.3849$$

$$\lambda_2 = 6.6151$$

Step 5: Computation of first principal components

$$e_1^T \begin{bmatrix} X_{1k} - \bar{X}_1 \\ X_{2k} - \bar{X}_2 \end{bmatrix}$$

F	Ex 1	Ex 2	Ex 3	Ex 4
X_1	4	8	13	7
X_2	11	4	5	14

$$e_1^T \begin{bmatrix} X_{1k} - \bar{X}_1 \\ X_{2k} - \bar{X}_2 \end{bmatrix} = \begin{bmatrix} 0.5574 & -0.8303 \end{bmatrix} \begin{bmatrix} X_{11} - \bar{X}_1 \\ X_{21} - \bar{X}_2 \end{bmatrix}$$

$$= 0.5574(X_{11} - \bar{X}_1) - 0.8303(X_{21} - \bar{X}_2)$$

$$= 0.5574(4 - 8) - 0.8303(11 - 8,5)$$

$$= -4.30535$$

$$e_1 = \begin{bmatrix} 0.5574 \\ -0.8303 \end{bmatrix} \qquad \overline{X_1} = 8$$

$$\overline{X_2} = 8.5$$

$$e_2 = \begin{bmatrix} 0.8303 \\ 0.5574 \end{bmatrix} \qquad S = \begin{bmatrix} 14 & -11 \\ -11 & 23 \end{bmatrix}$$

$$\lambda_1 = 30.3849$$

$$\lambda_2 = 6.6151$$

Now repeat for ex 2, ex3, ex 4 from above example.

Step 5: Computation of first principal components

F	Ex 1	Ex 2	Ex 3	Ex 4
X_1	4	8	13	7
X_2	11	4	5	14

Feature	Ex 1	Ex 2	Ex 3	Ex 4
X_1	4	8	13	7
X_2	11	4	5	14
First Principle Components	-4.3052	3.7361	5.6928	-5.1238

$$\overline{X_1} = 8$$

$$\overline{X_2} = 8.5$$

$$S = \begin{bmatrix} 14 & -11 \\ -11 & 23 \end{bmatrix}$$

$$\lambda_1 = 30.3849$$

$$\lambda_2 = 6.6151$$

Factor Analysis:

Factor analysis is one of the unsupervised machine learning algorithms which is used for dimensionality reduction. This algorithm creates factors from the observed variables to represent the common variance i.e. variance due to correlation among the observed variables.

$$x_i - \mu_i = l_1 F_1 + \cdots + l_n F_n + \varepsilon_i$$

Representing features in terms of factors ***x*** is the variable and ***F*** is the factor and **l** is the factor loading which can also be considered as the weight of the factor for the corresponding variable. The number of factors is equal to the number of variables.

The main aim of the factor analysis is to find the intercorrelations among n variables through a set of common factors (the number of factors is less than the n variables). In simple terms, it groups the variables into meaningful categories.

- We should apply factor Analysis on the **large datasets** to diminish the error in the final results. There should be **high factor loading scores** (>0.80) to use the factor analysis on a small dataset.
- The *correlation* between the factors and variables should be **at least 0.30,** as anything lower than this would imply a weak relationship between the variables.
- The *SMC(Squared Multiple Correlation)* of the dataset needs to be checked, and variables that have issues with a singularity, i.e., **SMC close to 0** and multicollinearity, i.e., **SMC close to 1.0,** should be **removed** from your dataset.
- We should also remove o**utliers** from the dataset.
- The dataset should be **standard scaled,** and we should convert the categorical features to numerical features.

Implementation of Factor Analysis

The various steps involved in factor analysis are:

1. *Checking the factorability of factor analysis*
2. *Determining the number of factors*
3. *Interpreting the factors*

Factorability of factor analysis

The methods to check the factorability are

1. *Correlation Matrix Check*
2. *KMO MSA Check*
3. *Bartlett's Test of Sphericity*

Correlation Matrix Check

Is the given dataset a combination of high and low correlations? If yes, then we can proceed with the factor analysis.

KMO MSA Check

The **Kaiser-Meyer-Olkin Measure of Sampling Adequacy** tests whether the partial correlations among variables are minor. It is a statistic that specifies the proportions of variance in the variables caused by underlying factors. High values, i.e., close to 1, indicate that the factor analysis is helpful for the dataset. If the value is less than 0.5, one shouldn't proceed with the factor analysis.

Bartlett's Test of Sphericity

It tests the hypothesis that the correlation matrix is an identity matrix. It will further indicate that the variables are unrelated, and therefore, factor analysis is not applicable here. If the value is less than 0.05, you can go ahead with the factor analysis. Essentially it checks to see a certain redundancy between the variables that we can summarize with a few factors. The **null hypothesis** of the test is that the **variables are orthogonal**, i.e., not correlated.

Determining the number of factors

If we extract too many factors, then undesirable error variance might occur. On the other hand, removing a handful of factors might leave out valuable common variance. So it's essential to select the most feasible way to decide the number of factors to extract.